

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e5140909-890\agent-a93779ccd36afc731.jsonl`  
- SHA-256: `c147a3ca7b2b98ac9f9234401b69601888d01a49fe24464ee4e0fc912008d32f`  
- Source modified: `2026-06-18T07:50:55+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_e5140909-890`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T07:47:49.191Z)

Work READ-ONLY in this git worktree pinned to origin/master (DO NOT modify):

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base  
Read the ACTUAL code (and the matching tests/ file) before asserting anything; never guess.  
Adversarial bug-hunter. Report ONLY real, reachable defects (crash on a reachable input, wrong  
logic, inverted condition, bad error handling, resource leak, off-by-one, contract mismatch,  
unguarded external/persisted-data access). Exact file+line + concrete trigger + minimal fix +  
confidence. PRECISION over recall ? empty list is a respected answer; do NOT invent findings.  
The repo has ~855 passing tests + several hardening sweeps; expect few. Skip pure style.  
AREA: backend/pipeline/detectors/ ? wasb_infer.py, tracknet_runner.py (R3/R4/R5 windowing:  
_trim_inactive_tail, _blob_fallback_chop, caps), person_detector.py, fusion_features.py,  
wasb_runner.py. Frame/time conversions, off-by-one, array bounds, GPU/np guards.
```

2. assistant (2026-06-18T07:47:51.584Z)

I'll start by reading the target files in the detectors directory and their matching tests.

3. user (2026-06-18T07:47:53.517Z)

```
total 144  
drwxr-xr-x 1 avidu 197609    0 Jun 18 13:15 .  
drwxr-xr-x 1 avidu 197609    0 Jun 18 13:15 ..  
-rw-r--r-- 1 avidu 197609 1067 Jun 18 13:15 __init__.py  
-rw-r--r-- 1 avidu 197609 1656 Jun 18 13:15 audio_features.py  
-rw-r--r-- 1 avidu 197609 9600 Jun 18 13:15 base.py  
-rw-r--r-- 1 avidu 197609 10057 Jun 18 13:15 fusion_features.py  
-rw-r--r-- 1 avidu 197609 17392 Jun 18 13:15 person_detector.py  
-rw-r--r-- 1 avidu 197609 4870 Jun 18 13:15 rally_gate.py  
-rw-r--r-- 1 avidu 197609 25784 Jun 18 13:15 tracknet_runner.py  
-rw-r--r-- 1 avidu 197609 33296 Jun 18 13:15 wasb_infer.py  
-rw-r--r-- 1 avidu 197609 10652 Jun 18 13:15 wasb_runner.py  
---TESTS---  
./tests/test_calibrate_wasb.py  
./tests/test_config_wasb.py  
./tests/test_detector_keying.py  
./tests/test_export_wasb_segments.py  
./tests/test_fusion_compare.py  
./tests/test_fusion_features.py  
./tests/test_fusion_golden_telemetry.py  
./tests/test_fusion_hybrid.py  
./tests/test_hit_detector.py  
./tests/test_personalization_serving.py  
./tests/test_person_detector.py  
./tests/test_tracknet_hybrid.py  
./tests/test_tracknet_runner.py
```

```
./tests/test_wasb_cache.py
./tests/test_wasb_runner.py
```

4. assistant (2026-06-18T07:47:55.903Z)

Let me read the main target files. I'll start with the windowing logic files (tracknet_runner.py R3/R4/R5) and the others.

5. user (2026-06-18T07:47:57.283Z)

```
1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14      Nothing here imports TensorFlow; all model code stays in WSL.
15      """
16
17      import os
18      import csv
19      import shlex
20      import logging
21      import subprocess
22      from dataclasses import dataclass
23      from typing import List, Dict, Any, Optional, Tuple
24
25      from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27      logger = logging.getLogger(__name__)
28
29
30      @dataclass
31      class TrackNetConfig:
32          """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33          distro: str = "Ubuntu"
34          repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36          conda_env: str = "TrackNetV4"
37          weights_path: str = ""                        # WSL path to the .h5/.keras weights
(REQUIRED)
38          queue_length: int = 5
39          stage_in_wsl: bool = True                     # copy video into WSL fs first
(perf)
40          wsl_stage_dir: str = "~/clips"                # where staged videos/outputs live
in WSL
41          timeout_sec: int = 1800                       # 30 min; kills a hung call (0 = no
timeout)
42          keep_frames: bool = False                     # retain staged video after success
(debug only)
43          min_free_gb: float = 0.0                     # warn if WSL free space below this
(0 = off)
44
45      @classmethod
46      def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47          tn = (idx_cfg or {}).get("tracknet", {}) or {}
```

```

48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63 def to_wsl_mnt_path(win_path: str) -> str:
64     """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66     Already-POSIX paths (starting with / or ~) are returned unchanged so the
67     same helper is safe to call on either kind of path.
68     """
69     p = str(win_path)
70     if p.startswith("/") or p.startswith("~"):
71         return p
72     p = p.replace("\\", "/")
73     if len(p) >= 2 and p[1] == ":":
74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             len(tf.config.list_physical_devices('GPU')))\\"
107         )
108         try:
109             res = self._wsl_bash(cmd)
110         except FileNotFoundError:
111             return False, "wsl` not found ? is this running on Windows with WSL2 installed?"

```

```

111         except subprocess.TimeoutExpired:
112             return False, "WSL healthcheck timed out."
113         out = (res.stdout or "").strip()
114         if res.returncode != 0:
115             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
116         return True, out
117
118     # ----- inference
119
120     def run_predict(self, video_win_path: str, output_win_dir: str,
121                    video_id: Optional[str] = None) -> Optional[str]:
122         """Run predict.py on a video. Returns the Windows path to the produced CSV, or
123         None.
124
125         ``output_win_dir`` is a Windows directory (under the repo's output/) that
126         WSL writes into via its /mnt view, so the windows process can read the CSV
127         back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
128         therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
129         a filename cannot collide on the output CSV.
130
131         """
132         if not self.cfg.weights_path:
133             logger.error(
134                 "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
135                 "in config.json to the WSL path of your trained TrackNetV4 weights."
136             )
137             return None
138
139         # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
140         # relative paths through unchanged, so WSL would resolve them against the
141         # predict.py cwd instead of the repo (same bug class as wasb_runner).
142         output_win_dir = os.path.abspath(output_win_dir)
143         os.makedirs(output_win_dir, exist_ok=True)
144         # Normalize separators so basename/splitext work when tests (or collector)
145         # pass windows-style paths on a Linux host.
146         video_win_path = str(video_win_path).replace("\\", "/")
147         base = os.path.basename(video_win_path)
148         stem, ext = os.path.splitext(base)
149
150         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
151         if ext.lower() not in (".mp4", ".avi"):
152             logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
153             return None
154
155         wsl_out = to_wsl_mnt_path(output_win_dir)
156         # Namespace by video_id so two same-filename videos don't collide on the CSV.
157         # predict.py derives its output name from the (staged) video stem, so staging
158         # under the namespaced name propagates into "<key>_predict.csv".
159         key = f"{stem}_{video_id[:12]}" if video_id else stem
160         expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
161
162         # Resolve the video path WSL will read.
163         if self.cfg.stage_in_wsl:
164             staged = self._stage_video(video_win_path, f"{key}{ext}")
165             if staged is None:
166                 return None
167             wsl_video = staged
168         else:
169             # No staging: predict.py reads /mnt directly and writes
170             # "<stem>_predict.csv".
171             # We cannot rename its output, so fall back to the un-namespaced name.
172             wsl_video = to_wsl_mnt_path(video_win_path)
173             expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
174
175         if self.cfg.min_free_gb > 0:
176             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)

```

```

174         if free is not None and free < self.cfg.min_free_gb:
175             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
176                             "consider running tools/cleanup_caches.py.",
177                             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
178
179         cmd = (
180             f"conda activate {self.cfg.conda_env} && "
181             f"cd {self.cfg.repo_dir}/src && "
182             f"python predict.py "
183             f"--video_path {wsl_tilde_quote(wsl_video)} "
184             f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
185             f"--output_dir {wsl_tilde_quote(wsl_out)} "
186             f"--queue_length {self.cfg.queue_length}"
187         )
188         logger.info("Running TrackNet inference on %s ...", base)
189         try:
190             res = self._wsl_bash(cmd)
191         except subprocess.TimeoutExpired:
192             logger.error("TrackNet inference timed out after %ss.",
193 self.cfg.timeout_sec)
194             return None
195
196         if res.returncode != 0:
197             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
198 res.stdout)[-2000:])
199             return None
200
201         if not os.path.exists(expected_csv_win):
202             logger.error("TrackNet finished but expected CSV not found at %s",
203 expected_csv_win)
204             return None
205         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
206
207         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
208         # regenerable intermediate). On earlier failure we return above without
209         cleaning.
210         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
211             logger.info("Cache hygiene: removing staged video for %s "
212                             "(set indexing.tracknet.keep_frames=true to retain).", key)
213             self._wsl_rm_rf([wsl_video])
214             return expected_csv_win
215
216     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
217         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
218
219         Returns the WSL path to the staged copy, or None on failure.
220         """
221         src_mnt = to_wsl_mnt_path(video_win_path)
222         reason = self.wsl_source_unreadable_reason(src_mnt)
223         if reason:
224             logger.error("Cannot stage video into WSL: %s", reason)
225             return None
226         dest = f"{self.cfg.wsl_stage_dir}/{base}"
227         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
228 {wsl_tilde_quote(dest)} && echo OK"
229         try:
230             res = self._wsl_bash(cmd)
231         except subprocess.TimeoutExpired:
232             logger.error("Staging video into WSL timed out.")
233             return None
234         if res.returncode != 0 or "OK" not in (res.stdout or ""):
235             logger.error("Failed to stage video into WSL: %s", (res.stderr or
236 res.stdout).strip())
237             return None
238         return dest

```

```

233
234 # ----- CSV -> windows
235
236 @staticmethod
237 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238     """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239     points: List[TrajectoryPoint] = []
240     with open(csv_win_path, newline="") as f:
241         reader = csv.DictReader(f)
242         for row in reader:
243             try:
244                 points.append(TrajectoryPoint(
245                     frame=int(row["Frame"]),
246                     visible=int(row["Visibility"]) == 1,
247                     x=float(row["X"]),
248                     y=float(row["Y"]),
249                 ))
250             except (KeyError, ValueError):
251                 continue
252     points.sort(key=lambda p: p.frame)
253     return points
254
255 @staticmethod
256 def trajectory_to_action_windows(
257     points: List[TrajectoryPoint],
258     fps: float,
259     frame_width: float,
260     chunk_start: float = 0.0,
261     velocity_thresh: float = 0.02,
262     merge_gap: float = 2.0,
263     min_window_duration: float = 2.0,
264     chunk_index: Optional[int] = None,
265     max_window_duration: float = 0.0,
266     min_split_gap: float = 0.5,
267     window_hard_cap: bool = False,
268     window_inactivity_end: float = 0.0,
269     inactivity_min_density: float = 0.34,
270     inactivity_rally_thresh: float = 0.08,
271     window_blob_fallback: bool = False,
272     window_blob_factor: float = 4.0,
273 ) -> List[Dict[str, Any]]:
274     """Convert a shuttle trajectory into candidate rally windows.
275
276     A frame is "active" when the shuttle is visible AND its inter-frame
277     displacement (normalised by frame width, per second) exceeds
278     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
279     ``merge_gap`` seconds of each other are grouped into one window; short
280     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
281     shape feeding the AI-handoff phase is identical.
282
283     ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
284     against the *over-merge* failure where "shuttle moving" is conflated with "rally in
285     progress" and a single window swallows several rallies + the dead time between them. When
286     set, any window longer than this is recursively SPLIT at its largest internal inactivity
287     gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
288     provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
289     fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
290     recall cannot drop, and it is config-gated default-OFF until measured on the golden set.

```

```

291
292     ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
an
293     over-long window has NO internal inactivity gap to split on (a continuously-
active
294     trajectory ? the shuttle never stops moving, e.g. the real-footage
``mahadevpura-1``
295     blob where the gap-splitter alone left a single 174 s window), it is recursively
296     hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
cuts
297     (never merges), so recall cannot drop; gated default-OFF until measured.
298
299     ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
unlike
300     ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
every gap-less
301     over-cap window and over-segments normal clips), this runs AFTER the R4
inactivity trim and
302     force-chops ONLY a genuine *blob* ? a group still longer than
``window_blob_factor`` *
303     ``max_window_duration`` once the gap-split and the principled rally-end trim
have both had
304     their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally
is left
305     alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
logged, and
306     each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
307     rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
308     can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
309     ""
310     if fps <= 0:
311         fps = 30.0
312     if frame_width <= 0:
313         frame_width = 1280.0
314
315     # Compute per-frame normalised velocity from the last *visible* point.
316     active = [] # (frame_idx, velocity)
317     prev: Optional[TrajectoryPoint] = None
318     for p in points:
319         if not p.visible:
320             prev = None # break the trajectory; absent shuttle = not in flight
321             continue
322         if prev is not None:
323             dframes = p.frame - prev.frame
324             if dframes > 0:
325                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
326                 velocity = (dist / frame_width) * (fps / dframes)
327                 if velocity > velocity_thresh:
328                     active.append((p.frame, velocity))
329             prev = p
330
331     if not active:
332         return []
333
334     max_gap_frames = int(merge_gap * fps)
335
336     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
337         vels = [v for _, v in group]
338         return {
339             "start": group[0][0] / fps + chunk_start,
340             "end": group[-1][0] / fps + chunk_start,
341             "active_frame_count": len(group),

```

```

342         "peak_velocity": max(vels),
343         "mean_velocity": sum(vels) / len(vels),
344         "track_id_count": 1, # single shuttle; kept for window-shape parity
345         "chunk_index": chunk_index,
346     }
347
348     groups: List[List[Tuple[int, float]]] = []
349     group = [active[0]]
350     for af in active[1:]:
351         if af[0] - group[-1][0] <= max_gap_frames:
352             group.append(af)
353         else:
354             groups.append(group)
355             group = [af]
356     groups.append(group)
357
358     if max_window_duration > 0:
359         groups = TrackNetRunner._split_overlong_groups(
360             groups, int(max_window_duration * fps), int(min_split_gap * fps),
361             hard_cap=window_hard_cap)
362
363     # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
364     # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
365     # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
366     # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
367     # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
368     if window_inactivity_end > 0:
369         tw = int(window_inactivity_end * fps)
370         groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
371                                                         inactivity_rally_thresh) for g
in groups]
372
373     # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
374     # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
375     # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
376     # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
377     forced_flags = [False] * len(groups)
378     if window_blob_fallback and max_window_duration > 0:
379         groups, forced_flags = TrackNetRunner._blob_fallback_chop(
380             groups, int(max_window_duration * fps), window_blob_factor)
381
382     windows = []
383     for g, forced in zip(groups, forced_flags):
384         w = _finalize(g)
385         if forced:
386             w["forced_cut"] = True
387         windows.append(w)
388     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
389
390     @staticmethod
391     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
392                             min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
393         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
394         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity

```



```

windowing
395         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
396         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
397         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
398         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
399         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
400         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
401         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
402         if trail_frames <= 0 or len(group) < 2:
403             return group
404         frames = [f for f, _ in group]
405         fast = [1 if v > rally_thresh else 0 for _, v in group]
406         lo = 0
407         fast_sum = 0
408         last_dense = -1
409         for i in range(len(frames)):
410             fast_sum += fast[i]
411             while frames[lo] <= frames[i] - trail_frames:
412                 fast_sum -= fast[lo]
413                 lo += 1
414             win_count = i - lo + 1
415             if win_count > 0 and fast_sum / win_count >= min_density:
416                 last_dense = i
417         return group[: last_dense + 1] if last_dense >= 0 else group
418
419     @staticmethod
420     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
421                               min_split_frames: int,
422                               hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
423         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
424         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
425         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
426         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
427         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
428         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
429         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
430         if max_win_frames <= 0:
431             return groups
432         out: List[List[Tuple[int, float]]] = []
433         work = list(groups)
434         while work:
435             g = work.pop()
436             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
437                 out.append(g)
438                 continue
439             best_i, best_gap = -1, -1
440             for i in range(1, len(g)):
441                 gap = g[i][0] - g[i - 1][0]
442                 if gap > best_gap:
443                     best_gap, best_i = gap, i
444             if best_i < 0 or best_gap < min_split_frames:

```

```

445         # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
446         # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
447         if hard_cap:
448             mid = (g[0][0] + g[-1][0]) / 2.0
449             split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
450             work.append(g[:split_i])
451             work.append(g[split_i:])
452         else:
453             out.append(g)
454             continue
455         work.append(g[:best_i])
456         work.append(g[best_i:])
457     out.sort(key=lambda grp: grp[0][0])
458     return out
459
460     @staticmethod
461     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
462                             blob_factor: float = 4.0
463                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
464         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
465         inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
466         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
467         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
468         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
469         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
470         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
471         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
472         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
473         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
474         that piece came from a forced chop. Iterative worklist to bound stack depth."""
475         if max_win_frames <= 0:
476             return groups, [False] * len(groups)
477         blob_frames = max_win_frames * max(1.0, blob_factor)
478         out: List[List[Tuple[int, float]]] = []
479         flags: List[bool] = []
480         forced = 0
481         for g in groups:
482             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
483                 out.append(g)
484                 flags.append(False)
485                 continue
486             pieces: List[List[Tuple[int, float]]] = []
487             work = [g]
488             while work:
489                 h = work.pop()
490                 if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
491                     pieces.append(h)
492                     continue
493                 mid = (h[0][0] + h[-1][0]) / 2.0
494                 split_i = min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid)) #
noqa: B023
495                 work.append(h[:split_i])
496                 work.append(h[split_i:])
497             pieces.sort(key=lambda grp: grp[0][0])

```

```

498         out.extend(pieces)
499         flags.extend([True] * len(pieces))
500         forced += len(pieces) - 1
501     if forced:
502         logger.warning(
503             "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
504             "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
505             "motion), not a bigger cap", forced)
506     return out, flags
507

```

6. user (2026-06-18T07:47:57.650Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3     NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4     math is unit-testable in a GPU-free dev container. The detection (person boxes,
5     shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6     module only turns those signals into a handful of scale/translation-invariant
7     features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8     memorise this court/camera and fail to generalise off a small labelled set).
9
10    Conventions:
11    - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12    - `per_frame` is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13        index (so it aligns directly with the shuttle trajectory's `frame` ? no clip re-
timing).
14    - A shuttle point has `frame`, `visible`, `x`, `y` (pixels) ? the WASB
trajectory shape.
15    - `court_polygon` is normalised 0-1 `(x, y), ...)` or `None` (None ? no mask,
every
16        detection counts ? the player-count-only mode).
17    - `net` is `(axis 'x'|'y', position 0-1 normalised)` or `None` (None ? two-sided =
0.0).
18
19    Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20    floats in [0, 1] except `mean_player_motion` (a non-negative normalised speed).
21    """
22    from __future__ import annotations
23
24    from typing import Dict, List, Optional, Sequence, Tuple
25
26    BBox = Tuple[float, float, float, float]
27    Net = Tuple[str, float]
28
29
30    def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31        """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32        (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33        if polygon is None or len(polygon) < 3:
34            return False
35        x, y = point
36        inside = False
37        n = len(polygon)
38        j = n - 1
39        for i in range(n):
40            xi, yi = polygon[i]
41            xj, yj = polygon[j]
42            # Does the horizontal ray at y cross edge (i, j)?

```

```

43         if (yi > y) != (yj > y):
44             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45             if x < x_cross:
46                 inside = not inside
47             j = i
48     return inside
49
50
51 def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52     """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53     the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54     if frame_width <= 0 or frame_height <= 0:
55         return (0.0, 0.0)
56     x1, _y1, x2, y2 = box
57     return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60 def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61     if frame_width <= 0 or frame_height <= 0:
62         return (0.0, 0.0)
63     x1, y1, x2, y2 = box
64     return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67 def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                    court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69     """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70     True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71     point meaningless, so a masked check returns False rather than testing a bogus
72     (0,0)."""
73     if court_polygon is None:
74         return True
75     if frame_width <= 0 or frame_height <= 0:
76         return False
77     return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
78
79 def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                   frame_height: float,
81                   court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82     """Per-frame count of in-court persons (one entry per sampled frame)."""
83     return [
84         sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
85         court_polygon))
86         for _frame_idx, dets in sorted(per_frame.items())
87     ]
88
89 def _median(vals: Sequence[float]) -> float:
90     if not vals:
91         return 0.0
92     s = sorted(vals)
93     n = len(s)
94     return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97 def players_in_court(counts: Sequence[int]) -> float:
98     """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99     return _median(list(counts))
100
101
102 def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:

```

```

103     """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
104     if not counts:
105         return 0.0
106     return sum(1 for c in counts if c >= min_players) / len(counts)
107
108
109 def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
110                        frame_width: float, frame_height: float) -> float:
111     """Mean per-track centre displacement between consecutive sampled frames, with x
112     normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
113     not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
114     if frame_width <= 0 or len(per_frame) < 2:
115         return 0.0
116     frames = sorted(per_frame.keys())
117     steps: List[float] = []
118     for a, b in zip(frames, frames[1:]):
119         prev = {tid: box for tid, box in per_frame[a]}
120         for tid, box in per_frame[b]:
121             if tid in prev:
122                 cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                 cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                 steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125     if not steps:
126         return 0.0
127     return sum(steps) / len(steps)
128
129
130 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131                     frame_height: float, net: Optional[Net],
132                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
137     0.0.
138     """
139     if net is None or not per_frame:
140         return 0.0
141     axis, pos = net
142     both = 0
143     for _frame_idx, dets in per_frame.items():
144         near, far = False, False
145         for _tid, box in dets:
146             if not person_in_court(box, frame_width, frame_height, court_polygon):
147                 continue
148             fx, fy = _foot_norm(box, frame_width, frame_height)
149             coord = fx if axis == "x" else fy
150             if coord < pos:
151                 near = True
152             else:
153                 far = True
154         if near and far:
155             both += 1
156     return both / len(per_frame)
157
158 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
159                    pad_frac: float) -> bool:
160     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
161     width/height (the 'adjacent / in-hand' tolerance)?"""
162     x1, y1, x2, y2 = box
163     px = pad_frac * frame_width
164     py = pad_frac * frame_height

```

```

165         return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
169                                     frame_width: float, frame_height: float,
170                                     pad_frac: float = 0.03) -> float:
171         """Fraction of (visible shuttle frames that were also person-sampled) in which the
172         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
173         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
174         """
175         if not per_frame or frame_width <= 0 or frame_height <= 0:
176             return 0.0
177         aligned = 0
178         colocated = 0
179         for p in shuttle_points or []:
180             if not getattr(p, "visible", False):
181                 continue
182             dets = per_frame.get(int(p.frame))
183             if dets is None:          # shuttle frame wasn't person-sampled ? can't judge
184                 continue
185             aligned += 1
186             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
187                     for _tid, box in dets):
188                 colocated += 1
189         return colocated / aligned if aligned else 0.0
190
191
192     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194     PERSON_FEATURE_NAMES = [
195         "players_in_court", "two_sided_motion", "mean_player_motion",
196         "in_court_ratio", "shuttle_player_colocation",
197     ]
198
199
200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
201                                     frame_width: float, frame_height: float,
202                                     court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
203                                     net: Optional[Net] = None,
204                                     min_players: int = 2) -> Dict[str, float]:
205         """Aggregate the per-window person features from per-frame detections + the in-
window
206         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
207         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
208         return {
209             "players_in_court": players_in_court(counts),
210             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
211             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
212             "in_court_ratio": in_court_ratio(counts, min_players),
213             "shuttle_player_colocation": shuttle_player_colocation(
214                 per_frame, shuttle_points, frame_width, frame_height),
215         }
216

```

7. assistant (2026-06-18T07:48:00.428Z)

Now let me read the remaining target files: wasb_infer.py, person_detector.py, and wasb_runner.py.

8. user (2026-06-18T07:48:01.816Z)

```

1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`,`trackers`,`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36         python wasb_infer.py --frames_dir /path/frames --weights
37         ../pretrained_weights/wasb_badminton_best.pth.tar \
38         --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39         every 1000] [--fresh]
40
41     """
42     import os
43     import os.path as osp
44     import csv
45     import glob
46     import json
47     import argparse
48     import logging
49     from collections import defaultdict
50     from contextlib import contextmanager
51
52     logger = logging.getLogger(__name__)
53
54     CACHE_VERSION = 1
55     _DET_FILE = "det_raw.jsonl"
56     _MANIFEST_FILE = "manifest.json"
57
58     def _build_cfg(weights: str, sport: str):
59         """Compose the WASB eval config via Hydra, overriding model/weights/device."""
60         from hydra import compose, initialize
61         with initialize(config_path="configs", version_base=None):
62             cfg = compose(config_name="eval", overrides=[
63                 f"dataset={sport}",
64                 "model=wasb",
65                 f"detector.model_path={weights}",

```

```

64         "runner.device=cuda",
65         "runner.gpus=[0]",          # this box has a single GPU; default config assumes
more
66     ])
67     return cfg
68
69
70     def _frame_id(path: str) -> int:
71         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
72         stem = osp.splitext(osp.basename(path))[0]
73         digits = "".join(ch for ch in stem if ch.isdigit())
74         return int(digits) if digits else -1
75
76
77     # ----- #
78     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
79     # ----- #
80     def _cache_paths(cache_dir: str):
81         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
82
83
84     def default_cache_dir(out_csv: str) -> str:
85         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
86         d = osp.dirname(osp.abspath(out_csv))
87         stem = osp.splitext(osp.basename(out_csv))[0]
88         return osp.join(d, stem + "_wasbcache")
89
90
91     def _atomic_write_text(path: str, text: str) -> None:
92         """Write then rename so a crash mid-write can't leave a half-written file."""
93         tmp = path + ".tmp"
94         with open(tmp, "w") as f:
95             f.write(text)
96             f.flush()
97             os.fsync(f.fileno())
98         os.replace(tmp, path)
99
100
101     def _det_plain(det) -> list:
102         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
103         xy = det["xy"]
104         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
105
106
107     def _dets_to_tracker(plain_list):
108         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
109         the tracker does vector math / np.linalg.norm on it)."""
110         import numpy as np
111         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
112                 for p in plain_list]
113
114
115     def write_manifest(cache_dir: str, manifest: dict) -> None:
116         _, mpath = _cache_paths(cache_dir)
117         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
118
119
120     def read_manifest(cache_dir: str):
121         _, mpath = _cache_paths(cache_dir)
122         if not osp.exists(mpath):
123             return None
124         try:

```



```

125         with open(mpath) as f:
126             return json.load(f)
127     except (json.JSONDecodeError, OSError):
128         return None
129
130
131     def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
132                             frames_in: int, frames_total: int) -> bool:
133         """A cache is reusable only if the run that produced it matches this run's
134         inputs."""
135         if not manifest or manifest.get("version") != CACHE_VERSION:
136             return False
137         return (
138             manifest.get("frames_dir") == osp.abspath(frames_dir)
139             and manifest.get("weights") == weights
140             and manifest.get("sport") == sport
141             and manifest.get("frames_in") == frames_in
142             and manifest.get("frames_total") == frames_total
143         )
144
145     def reset_cache(cache_dir: str) -> None:
146         """Drop any existing cache so the next run starts clean."""
147         det_jsonl, mpath = _cache_paths(cache_dir)
148         for p in (det_jsonl, mpath):
149             if osp.exists(p):
150                 os.remove(p)
151
152
153     def load_and_clean_cache(cache_dir: str):
154         """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
155         rebuild the per-frame detection accumulation, and rewrite the file to exactly that
156         prefix so a trailing half-written line from a crash can't corrupt future appends.
157
158         Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
159         [x, y, score, scale] candidates (accumulated across every window the frame is in,
160         in window order ? identical to a non-resumed run).
161         """
162         det_jsonl, _ = _cache_paths(cache_dir)
163         det_by_fid = defaultdict(list)
164         valid: list = []
165         if osp.exists(det_jsonl):
166             with open(det_jsonl, "r") as f:
167                 for line in f:
168                     s = line.strip()
169                     if not s:
170                         continue
171                     try:
172                         obj = json.loads(s)
173                     except json.JSONDecodeError:
174                         break # truncated trailing line (crash mid-
flush)
175                     if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
176                         break
177                     valid.append(s)
178                     for fid, plain in obj["f"]:
179                         det_by_fid[int(fid)].extend([list(p) for p in plain])
180                     # Guarantee a clean append boundary by rewriting only the good prefix.
181                     _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
182         return len(valid), det_by_fid
183
184
185     def _append_windows(fh, objs) -> None:
186         """Append window records as JSONL and durably flush (bounded loss on crash)."""
187         for o in objs:

```

```

188         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
189     fh.flush()
190     os.fsync(fh.fileno())
191
192
193 def _atomic_write_trajectory(out_csv: str, rows) -> None:
194     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
195     tmp = out_csv + ".tmp"
196     with open(tmp, "w", newline="") as f:
197         w = csv.writer(f)
198         w.writerow(["Frame", "Visibility", "X", "Y"])
199         w.writerows(rows)
200         f.flush()
201         os.fsync(f.fileno())
202     os.replace(tmp, out_csv)
203
204
205 # ----- #
206 # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
207 # ----- #
208 _STREAM_FRAME_FMT = "{:08d}.png"
209
210
211 def synthetic_frame_path(index: int) -> str:
212     """Stable, index-encoding frame name used by the streaming path.
213
214     It mirrors the on-disk names ``extract_frames`` writes (``{:08d}.png``) so the
215     downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
216     frames came from disk or from the in-memory stream. The streaming image loader
217     inverts this name back to an index to fetch the decoded frame.
218     """
219     return _STREAM_FRAME_FMT.format(int(index))
220
221
222 def build_windows(frames: list, frames_in: int):
223     """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
224     + checkpoint). Pure list math, identical for the disk and streaming paths.
225
226     Returns a list of windows, each a list of ``frames_in`` consecutive paths
227     (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
228     caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
229     ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
230     """
231     return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
232
233
234 class SequentialFrameStore:
235     """In-memory sliding buffer over a forward-only frame reader, sized for the
236     detector's sliding-window access pattern (peak O(window) frames, not O(video)).
237
238     The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
239     ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
240     in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
241     start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
242     boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
243     far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
244     earliest index any not-yet-finished window can still ask for) and decode each source
245     frame exactly once via the forward-only ``reader(index)``.
246     """
247
248     def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
249         self._reader = reader
250         self._total = int(total)
251         self._window = max(1, int(window))
252         self._buf: dict = {}

```

```

253         # On a resumed run the detector's first needed frame is `start_index`; begin
254         # pulling there so the reader can skip (seek past) the already-cached prefix
255         # instead of decoding 0..start_index-1 just to throw them away.
256         self._next = int(start_index)      # next index not yet pulled from the reader
257         self._max_seen = int(start_index) - 1
258         self._floor = int(start_index)      # lowest index we still keep (never evict
below)
259
260     def get(self, index: int):
261         index = int(index)
262         if index < self._floor:
263             raise KeyError(
264                 f"frame {index} already evicted (below floor {self._floor}); the access
"
265                 f"pattern must stay within a window of the highest frame seen")
266         if index >= self._total:
267             raise KeyError(f"frame {index} out of range (total {self._total})")
268         # Pull forward from the reader until the requested index is buffered.
269         while self._next <= index:
270             self._buf[self._next] = self._reader(self._next)
271             self._next += 1
272         if index > self._max_seen:
273             self._max_seen = index
274         # Evict frames older than the earliest a still-running window could re-request:
275         # once we've seen index `m`, no future window starts before `m - (window-1)`.
276         new_floor = max(self._floor, self._max_seen - (self._window - 1))
277         for k in range(self._floor, new_floor):
278             self._buf.pop(k, None)
279         self._floor = new_floor
280         return self._buf[index]
281
282
283     def _index_from_synthetic_path(path: str) -> int:
284         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
285
286         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
287         in lockstep with the on-disk naming scheme.
288         """
289         return _frame_id(path)
290
291
292     def _bgr_to_pil_rgb(frame_bgr):
293         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
294         PIL RGB image WASB's disk path produces.
295
296         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
297         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
298         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
299         frame indistinguishable from the on-disk one to everything downstream.
300         """
301         import cv2
302         from PIL import Image
303         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
304
305
306     def _make_streamed_read_image(store, real_read_image):
307         """Build the ``read_image`` replacement used during a streaming detector pass.
308
309         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
image,
310         matching ``read_image``'s real return type); any path that doesn't parse to an index

```

```

falls
311         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
testable
312         without the WSL-only ``dataloaders`` package.
313         """
314         def _read_image(path, *args, **kwargs):
315             idx = _index_from_synthetic_path(path)
316             if idx < 0:
317                 return real_read_image(path, *args, **kwargs)
318             return _bgr_to_pil_rgb(store.get(idx))
319         return _read_image
320
321
322     @contextmanager
323     def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
int = 0):
324         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
325         frames during the detector pass, byte-for-byte as it would over real PNGs.
326
327         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
328         ``cv2.imread``. ``read_image`` (``utils.utils``) does
329         ``Image.open(path).convert('RGB')``,
330         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
331         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
332         byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
333         ``osp.exists`` guard is sidestepped for synthetic stream paths.
334
335         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
336         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
337         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
338         reference).
339         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
340         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
341         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
342         original reader is restored on exit.
343         """
344         from dataloaders import dataset_loader as _dl
345         store = SequentialFrameStore(frame_loader, total, window=window,
start_index=start_index)
346         real_read_image = _dl.read_image
347         # Rebind read_image in the consuming module for the duration of the detector pass;
348         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
349         _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
ignore[assignment]
350         try:
351             yield store
352         finally:
353             _dl.read_image = real_read_image # type: ignore[assignment]
354
355
356     # ----- #
357     # Inference
358     # ----- #
359     def run(frames, weights: str, out_csv: str, sport: str = "badminton",

```

```

360         limit: int = 0, batch_size: int = 8, num_workers: int = 4,
361         log_every_batches: int = 50, cache_dir: "str | None" = None,
362         flush_every: int = 1000, fresh: bool = False,
363         frame_loader=None, source_key: "str | None" = None) -> str:
364     """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
365
366     Two equivalent ways to supply pixels (output is bit-identical between them):
367
368     * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
369       globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
370       ``cv2.imread``.
371     * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
372       frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
373       pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
374       reader, which returns a PIL RGB image) over the DataLoader pass so every other
375       line of ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
376       identical to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
377       ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces
378       ``num_workers=0``
379       (the in-process shim + buffer can't cross worker processes).
380
381     ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
382     frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
383     after reboot still matches.
384     """
385     import time
386     import torch
387     from torch.utils.data import DataLoader
388     from detectors import build_detector
389     from trackers import build_tracker
390     from dataloaders import build_img_transforms, build_seq_transforms
391     from dataloaders.dataset_loader import ImageDataset
392     from utils import Center
393
394     streaming = frame_loader is not None
395
396     cfg = _build_cfg(weights, sport)
397     frames_in = int(cfg["model"]["frames_in"])
398     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
399     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
400
401     if streaming:
402         # `frames` is already the ordered list of (synthetic) frame paths.
403         if not isinstance(frames, (list, tuple)):
404             raise SystemExit("streaming mode requires `frames` to be a list of frame
405             paths")
406         frames = list(frames)
407         frames_dir = source_key or "stream"
408     else:
409         frames_dir = frames
410         frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
411         glob.glob(osp.join(frames_dir, "*.jpg"))
412     if limit:
413         frames = frames[:limit]
414     if len(frames) < frames_in:
415         where = frames_dir if not streaming else "video stream"
416         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
417
418     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
419     checkpoint).
420     samples = []
421     for window in build_windows(frames, frames_in):
422         annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}]
```

```

for p in window]
418     samples.append({"frames": window, "annos": annos})
419     windows_total = len(samples)
420
421     # ---- cache setup / resume decision ----- #
422     if cache_dir is None:
423         cache_dir = default_cache_dir(out_csv)
424         os.makedirs(cache_dir, exist_ok=True)
425         det_jsonl, _ = _cache_paths(cache_dir)
426         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
427         manifest = None if fresh else read_manifest(cache_dir)
428         resume = manifest_compatible(
429             manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
430             frames_in=frames_in, frames_total=len(frames),
431         )
432         if resume:
433             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
434             logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
435         else:
436             if manifest is not None:
437                 logger.info("cache incompatible with this run -> starting fresh")
438                 reset_cache(cache_dir)
439                 windows_done, det_by_fid = 0, defaultdict(list)
440
441         base_manifest = {
442             "version": CACHE_VERSION,
443             "frames_dir": cache_key,
444             "weights": weights,
445             "sport": sport,
446             "frames_in": frames_in,
447             "frames_total": len(frames),
448             "windows_total": windows_total,
449             "windows_done": windows_done,
450         }
451         write_manifest(cache_dir, base_manifest)
452
453     # ---- detector pass over the un-cached windows ----- #
454     remaining = samples[windows_done:]
455     if remaining:
456         _, transform_test = build_img_transforms(cfg)
457         try:
458             _, seq_transform_test = build_seq_transforms(cfg)
459         except Exception:
460             seq_transform_test = None
461         ds = ImageDataset(cfg, remaining, input_wh, output_wh,
462             transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
463         # Streaming forces single-process loading: the in-memory frame store + the
464         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
465         loader_workers = 0 if streaming else num_workers
466         loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
467         detector = build_detector(cfg)
468
469         from contextlib import ExitStack
470
471         t0 = time.time()
472         done = windows_done
473         win_idx = windows_done
474         log_every = max(1, log_every_batches)
475         flush_n = max(1, flush_every)
476         pending = []
477         logger.info(f"detecting on {len(remaining)} remaining windows ")

```

```

478         f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
479         f"source={'video-stream' if streaming else 'frames-dir'})...")
480     det_f = open(det_jsonl, "a")
481     try:
482         with ExitStack() as stack:
483             stack.enter_context(torch.no_grad())
484             # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
485             # in-memory decoded frame for that synthetic path; every other line of
486             # __getitem__ runs unchanged so the tensor matches the PNG round-trip
exactly.
487             # The detector's first needed frame is `windows_done` (window i starts
at
488             # frame i), so prime the store there for a resume.
489             if streaming:
490                 stack.enter_context(
491                     _streaming_read_image_patch(frame_loader, len(frames),
492                                                 window=frames_in,
start_index=windows_done))
493             for bi, batch in enumerate(loader):
494                 imgs, _hms, trans = batch[0], batch[1], batch[2]
495                 img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
496                 batch_results, _ = detector.run_tensor(imgs, trans)
497                 nb = imgs.shape[0]
498                 for ib in range(nb):
499                     frames_payload = []
500                     for ie in sorted(batch_results[ib].keys()):
501                         p = img_paths[ie][ib]
502                         fid = _frame_id(p)
503                         plain = [_det_plain(d) for d in batch_results[ib][ie]]
504                         frames_payload.append([fid, plain])
505                         det_by_fid[fid].extend(plain)
506                     pending.append({"w": win_idx, "f": frames_payload})
507                     win_idx += 1
508                 done += nb
509                 if len(pending) >= flush_n or done >= windows_total:
510                     _append_windows(det_f, pending)
511                     pending = []
512                     base_manifest["windows_done"] = done
513                     write_manifest(cache_dir, base_manifest)
514                 if (bi + 1) % log_every == 0 or done >= windows_total:
515                     el = time.time() - t0
516                     new_done = done - windows_done
517                     rate = new_done / el if el > 0 else 0.0
518                     eta = (windows_total - done) / rate if rate > 0 else 0.0
519                     logger.info(f"{done}/{windows_total} "
520                                f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
521                                f"elapsed {el:.0f}s | ETA {eta:.0f}s")
522                 if pending:
523                     _append_windows(det_f, pending)
524                     base_manifest["windows_done"] = done
525                     write_manifest(cache_dir, base_manifest)
526             finally:
527                 det_f.close()
528                 logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
529         else:
530             logger.info("detector cache already complete -> tracker-only re-run")
531
532     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
533     tracker = build_tracker(cfg)
534     tracker.refresh()
535     rows = []

```

```

536     for fid in sorted(det_by_fid.keys()):
537         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
538         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
539
540     _atomic_write_trajectory(out_csv, rows)
541     visible = sum(1 for _, v, *_ in rows if v)
542     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
543     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
544     return out_csv
545
546
547 def extract_frames(video_path: str, frames_dir: str) -> str:
548     """Decode a video to PNG frames named by source frame index (00000000.png ...).
549
550     Idempotent: if the dir already holds the expected number of frames (matching the
551     container's reported frame count) we skip re-decoding ? re-extraction after a
552     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
553     extracting with ffmpeg on the host and passing --frames_dir.
554     """
555     import cv2
556     os.makedirs(frames_dir, exist_ok=True)
557     cap = cv2.VideoCapture(video_path)
558     if not cap.isOpened():
559         raise SystemExit(f"cannot open video: {video_path}")
560     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
561     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
562     if expected > 0 and existing >= expected:
563         cap.release()
564         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
565         return frames_dir
566     i = 0
567     while True:
568         ok, frame = cap.read()
569         if not ok:
570             break
571         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
572         i += 1
573     cap.release()
574     logger.info(f"extracted {i} frames -> {frames_dir}")
575     return frames_dir
576
577
578 def _probe_frame_count(video_path: str) -> int:
579     """Container-reported frame count via cv2 (0 if unknown). Used to size the
580     stream."""
581     import cv2
582     cap = cv2.VideoCapture(video_path)
583     if not cap.isOpened():
584         raise SystemExit(f"cannot open video: {video_path}")
585     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
586     cap.release()
587     return n
588
589 def make_video_frame_loader(video_path: str):
590     """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
591
592     ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
593     long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
594     detector's sliding-window order); it reads sequentially and only uses a frame-seek
595     to
596     skip forward when a *resume* starts past the current position. Each ``cap.read()``
597     returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
598     would
599     yield (PNG is lossless), so the detector sees identical pixels to the disk path.

```



```

598
599 Returns the count hint from the container so the caller can build the frame list;
600 ``release`` closes the capture.
601 """
602 import cv2
603 cap = cv2.VideoCapture(video_path)
604 if not cap.isOpened():
605     raise SystemExit(f"cannot open video: {video_path}")
606 count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
607 state = {"pos": 0} # next index cap.read() will return
608
609 def frame_loader(index: int):
610     index = int(index)
611     if index < state["pos"]:
612         raise RuntimeError(
613             f"streaming decode is forward-only; got index {index} after
{state['pos']}")
614     if index > state["pos"]:
615         # Forward skip only happens when resuming past cached windows. Seek once.
616         cap.set(cv2.CAP_PROP_POS_FRAMES, index)
617         state["pos"] = index
618     ok, frame = cap.read()
619     if not ok:
620         raise RuntimeError(f"failed to decode frame {index} from {video_path}")
621     state["pos"] += 1
622     return frame
623
624 def release():
625     cap.release()
626
627 return frame_loader, count_hint, release
628
629
630 def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
631                        limit: int = 0, batch_size: int = 8,
632                        log_every_batches: int = 50, cache_dir: str | None = None,
633                        flush_every: int = 1000, fresh: bool = False) -> str:
634     """Fast path: decode ``video_path`` on the fly and feed frames straight to the
635     detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
636     dominates a long clip). Output is bit-identical to the frames-on-disk path; see
637     ``run``'s docstring for why.
638     """
639     frame_loader, count_hint, release = make_video_frame_loader(video_path)
640     try:
641         if count_hint <= 0:
642             raise SystemExit(
643                 f"could not determine frame count for {video_path}; cannot stream "
644                 f"(fall back to frame extraction with --frames_dir)")
645         frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
646         source_key = "video:" + osp.abspath(video_path)
647         return run(frame_list, weights, out_csv, sport, limit,
648                   batch_size=batch_size, num_workers=0,
649                   log_every_batches=log_every_batches, cache_dir=cache_dir,
650                   flush_every=flush_every, fresh=fresh,
651                   frame_loader=frame_loader, source_key=source_key)
652     finally:
653         release()
654
655
656 def main():
657     ap = argparse.ArgumentParser()
658     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
659     ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")

```

```

660     ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
661     ap.add_argument("--stream-video", action="store_true",
662                     help="FAST PATH: decode --video on the fly and feed frames straight
to "
663                         "the detector (no PNG extraction). Output-equivalent to the
disk path.")
664     ap.add_argument("--weights", required=True)
665     ap.add_argument("--out", required=True)
666     ap.add_argument("--sport", default="badminton")
667     ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
668     ap.add_argument("--batch-size", type=int, default=8)
669     ap.add_argument("--num-workers", type=int, default=4)
670     ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
671     ap.add_argument("--cache-dir", default=None,
672                     help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
673     ap.add_argument("--flush-every", type=int, default=1000,
674                     help="fsync the detector cache every N windows (bounds reboot
loss)")
675     ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
676     args = ap.parse_args()
677
678     if args.stream_video:
679         if not args.video:
680             raise SystemExit("--stream-video requires --video")
681         run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
682                             batch_size=args.batch_size,
683                             log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
684                             flush_every=args.flush_every, fresh=args.fresh)
685         return
686
687     frames_dir = args.frames_dir
688     if args.video:
689         frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
690     if not frames_dir:
691         raise SystemExit("provide --frames_dir or --video")
692     run(frames_dir, args.weights, args.out, args.sport, args.limit,
693         batch_size=args.batch_size, num_workers=args.num_workers,
694         log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
695         flush_every=args.flush_every, fresh=args.fresh)
696
697
698     if __name__ == "__main__":
699         main()
700

```

9. user (2026-06-18T07:48:01.893Z)

```

1     """Person-detection rally cue ? torchvision detector + ByteTrack + motion windowing.
2
3     Extracted verbatim from `yolo_hybrid` (fusion P1, `docs/MULTI_SIGNAL_FUSION_PLAN.md`):
4     the **person cue is a swappable producer**, not logic baked into one segmenter. Today
5     `HybridYoloSegmenter` is the only consumer (it delegates Stage-1 here, byte-
identically);
6     next the fusion segmenter consumes the SAME producer over shuttle-seeded windows, and
7     later the learned classifier reads its persisted features. Keeping detection here (not
in
8     the segmenter) is what makes "add a cue = register a producer" true.
9

```

```

10 Behaviour is unchanged from the in-lined version ? only the home moved:
11 - `lazy_load_model` ? load a permissively-licensed torchvision detector
12 - `make_tracker` ? a fresh supervision ByteTrack per chunk
13 - `detect_and_track` ? one frame ? confident persons with persistent IDs
14 - `extract_action_windows_from_chunk` ? a chunk ? high-motion candidate windows
15 with the motion stats (`peak/mean_velocity`, `active_frame_count`, `track_id_count`)
16 that feed the candidate `stats` JSON and the learned fusion features.
17
18 The model libs (torchvision, supervision) are imported lazily inside the methods, and
the
19 detector weights load only on first detection (`lazy_load_model`) ? so importing the
20 segmenter registry never loads a detector model, and tests can mock the seams. (torch +
cv2
21 are top-level imports here; CI's import-smoke stubs them, and they're present on the GPU
22 machine.) Licensing: torchvision detectors are BSD + COCO weights; ByteTrack is MIT
23 (ADR-005; ultralytics' AGPL YOLO was dropped).
24 """
25 import logging
26 from typing import Any, Callable, Dict, List, Optional, Tuple
27
28 import cv2
29 import torch
30
31 from backend.utils.geometry import get_velocity_normalised
32
33 logger = logging.getLogger(__name__)
34
35 # torchvision detection models are trained on COCO where the "person" category is
36 # class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
37 # this off-by-one is the classic gotcha when swapping detectors. Keep it named.
38 _PERSON_CLASS_ID = 1
39
40 # Permissively-licensed torchvision detectors selectable via config
41 # `indexing.detector_model`. All are BSD-licensed (code + COCO weights).
42 _DETECTOR_FACTORIES = {
43     "fasterrcnn_resnet50_fpn": "fasterrcnn_resnet50_fpn",
44     "fasterrcnn_mobilenet_v3_large_fpn": "fasterrcnn_mobilenet_v3_large_fpn",
45     "retinanet_resnet50_fpn": "retinanet_resnet50_fpn",
46 }
47 _DEFAULT_DETECTOR = "fasterrcnn_resnet50_fpn"
48
49
50 class PersonDetector:
51     """Stage-1 person detection + tracking + motion-window extraction (a rally cue).
52
53     Owns the torchvision model + per-chunk ByteTrack tracker. Constructed with the run
54     ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
55     min_action_window_duration}``); the model loads lazily on first use.
56     """
57
58     def __init__(self, config: Dict[str, Any]):
59         self.config = config
60         # Dynamic torchvision module (None until lazy_load_model); typed Any so the
61         # `self.model([tensor])` inference call type-checks without a quarantine.
62         self.model: Any = None
63         self.device: Optional[str] = None
64
65     def lazy_load_model(self) -> None:
66         if self.model is not None:
67             return
68
69         # Imported lazily so the module imports without torchvision present and so
70         # the test suite (which pre-sets self.model) never pulls in model weights.
71         from torchvision.models import detection as tv_detection
72

```

```

73         idx_cfg = self.config.get("indexing", {})
74         use_gpu = idx_cfg.get("use_gpu", True)
75         self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
76         if use_gpu and not torch.cuda.is_available():
77             logger.warning("use_gpu is True but CUDA is not available. Falling back to
CPU.")
78
79         model_name = idx_cfg.get("detector_model", _DEFAULT_DETECTOR)
80         if model_name not in _DETECTOR_FACTORIES:
81             logger.warning(f"Unknown detector_model '{model_name}'. Falling back to
{_DEFAULT_DETECTOR}.")
82             model_name = _DEFAULT_DETECTOR
83
84         builder = getattr(tv_detection, _DETECTOR_FACTORIES[model_name])
85         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
86         # weights="DEFAULT" pulls the best available COCO-pretrained checkpoint.
87         self.model = builder(weights="DEFAULT")
88         self.model.eval()
89         self.model.to(self.device)
90
91     def make_tracker(self, fps: float) -> Any:
92         """Create a fresh ByteTrack tracker for one chunk (track IDs are chunk-local).
93
94         Imported lazily so tests can mock this method without supervision installed.
95         """
96         import supervision as sv
97         frame_rate = max(1, int(round(fps)))
98         return sv.ByteTrack(frame_rate=frame_rate)
99
100     def detect_and_track(self, frame: Any, tracker: Any,
101                          score_thresh: float) -> List[Tuple[int, Tuple[float, float,
float, float]]]:
102         """Detect persons in a frame and assign persistent track IDs.
103
104         Returns a list of (track_id, bbox) where bbox is (x1, y1, x2, y2) in pixels.
105         This replaces the old ultralytics ``model.track(...)`` call, which bundled
106         detection AND association: torchvision does the detection, supervision's
107         ByteTrack does the association.
108         """
109         import supervision as sv
110
111         # cv2 frames are BGR uint8 HxWxC; torchvision wants RGB float [0,1] CxHxW.
112         rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
113         tensor = torch.from_numpy(rgb).permute(2, 0,
1).float().div(255.0).to(self.device)
114         with torch.no_grad():
115             outputs = self.model([tensor])
116
117         out = outputs[0]
118         boxes = out["boxes"].detach().cpu().numpy()
119         labels = out["labels"].detach().cpu().numpy()
120         scores = out["scores"].detach().cpu().numpy()
121
122         # Keep only confident person detections.
123         mask = (labels == _PERSON_CLASS_ID) & (scores >= score_thresh)
124         if not mask.any():
125             return []
126
127         # ByteTrack requires confidence to be populated (it filters on it internally).
128         dets = sv.Detections(xyxy=boxes[mask], confidence=scores[mask],
class_id=labels[mask])
129         tracked = tracker.update_with_detections(dets)
130
131         result: List[Tuple[int, Tuple[float, float, float, float]]] = []
132         for i in range(len(tracked)):

```

```

133         tid = tracked.tracker_id[i]
134         if tid is None:
135             continue
136         x1, y1, x2, y2 = tracked.xyxy[i]
137         result.append((int(tid), (float(x1), float(y1), float(x2), float(y2))))
138     return result
139
140     def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
141                                         velocity_thresh: float, merge_gap: float,
142                                         frame_skip: int = 3,
143                                         chunk_index: Optional[int] = None) ->
144     List[Dict[str, Any]]:
145         """
146         Runs person detection + tracking on a video chunk and extracts high-motion
147         action windows. Returns a list of dicts in the full video's timeframe, each
148         with:
149             start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
150             track_id_count, chunk_index.
151
152         Args:
153             frame_skip: Process every Nth frame (1 = every frame). Higher values
154                         dramatically reduce inference cost with minimal quality loss.
155             chunk_index: Index of the source chunk (recorded on each window for
156                         traceability).
157         """
158         cap = cv2.VideoCapture(chunk_path)
159         if not cap.isOpened():
160             logger.error(f"Could not open {chunk_path}")
161             return []
162
163         fps = cap.get(cv2.CAP_PROP_FPS)
164         if fps <= 0:
165             fps = 30.0
166
167         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
168         if frame_width <= 0:
169             frame_width = 1280.0 # Sensible fallback
170
171         # Effective sampling rate after frame-skipping
172         effective_fps = fps / max(frame_skip, 1)
173
174         # A fresh tracker per chunk ? track IDs only need to be consistent within a
175         # chunk (windows are computed per chunk, then merged across chunks by time).
176         tracker = self.make_tracker(effective_fps)
177         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
178         0.5)
179
180         # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
181         # so windows can be enriched with motion stats for logging + fine-tuning.
182         active_frames = []
183         frame_idx = 0
184         track_history: Dict[int, Tuple[float, float, float, float]] = {}
185
186         while True:
187             ret, frame = cap.read()
188             if not ret:
189                 break
190
191             # Skip frames for performance ? at 30fps, every 3rd frame still
192             # gives ~10 samples/second, plenty for human-scale velocity.
193             if frame_idx % max(frame_skip, 1) != 0:
194                 frame_idx += 1
195                 continue
196
197             # Detect persons + assign persistent track IDs.

```

```

194         detections = self.detect_and_track(frame, tracker, score_thresh)
195
196         frame_peak_velocity = 0.0
197         frame_active_ids = set()
198         current_ids = set()
199
200         for track_id, bbox in detections:
201             current_ids.add(track_id)
202
203             if track_id in track_history:
204                 prev_bbox = track_history[track_id]
205                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
206
207                 if velocity > velocity_thresh:
208                     frame_peak_velocity = max(frame_peak_velocity, velocity)
209                     frame_active_ids.add(track_id)
210
211                 track_history[track_id] = bbox
212
213             # Prune stale IDs that are no longer visible ? prevents false
214             # velocity spikes when the tracker reassigns an old ID.
215             track_history = {k: v for k, v in track_history.items() if k in current_ids}
216
217             if frame_active_ids:
218                 active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
219
220             frame_idx += 1
221
222         cap.release()
223
224         # Group active frames into continuous windows
225         if not active_frames:
226             return []
227
228         max_gap_frames = int(merge_gap * fps)
229
230         def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
231             velocities = [v for _, v, _ in group]
232             track_ids = set().union(*[ids for _, _, ids in group])
233             return {
234                 "start": group[0][0] / fps + chunk_start,
235                 "end": group[-1][0] / fps + chunk_start,
236                 "active_frame_count": len(group),
237                 "peak_velocity": max(velocities),
238                 "mean_velocity": sum(velocities) / len(velocities),
239                 "track_id_count": len(track_ids),
240                 "chunk_index": chunk_index,
241             }
242
243         windows = []
244         group = [active_frames[0]]
245         for af in active_frames[1:]:
246             if af[0] - group[-1][0] <= max_gap_frames:
247                 group.append(af)
248             else:
249                 windows.append(_finalize(group))
250                 group = [af]
251         windows.append(_finalize(group))
252
253         # Filter out extremely short action windows
254         min_window_duration = self.config.get("indexing",
{}).get("min_action_window_duration", 2.0)
255         windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
256
257         return windows

```

```

257
258     def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
259                               frame_skip: int = 3) -> Dict[str, Any]:
260         """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
261         video.
262         Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
263         ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
264         discards the
265         per-frame boxes), this returns the raw per-frame detections so the fusion
266         features can
267         align them with the shuttle trajectory. It reads the source by **absolute frame
268         index**
269         (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
270         match
271         the WASB trajectory's ``.frame`` exactly ? that alignment is what
272         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
273         Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
274         "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
275         seek
276         accuracy varies by codec (it lands near the requested keyframe then decodes
277         forward);
278         treat the indices as ±a few frames on real footage ? fine for window-level
279         features.
280         """
281         self.lazy_load_model()
282         cap = cv2.VideoCapture(video_path)
283         if not cap.isOpened():
284             logger.error(f"Could not open {video_path}")
285             return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
286
287         fps = cap.get(cv2.CAP_PROP_FPS)
288         if fps <= 0:
289             fps = 30.0
290         frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
291         frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
292         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
293         0.5)
294         step = max(frame_skip, 1)
295         tracker = self.make_tracker(fps / step)
296
297         start_frame = max(0, int(start_frame))
298         frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
299         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
300         frame_idx = start_frame
301         while frame_idx <= end_frame:
302             ret, frame = cap.read()
303             if not ret:
304                 break
305             if (frame_idx - start_frame) % step == 0:
306                 frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
307                 frame_idx += 1
308         cap.release()
309         return {"frames": frames, "fps": fps,
310                 "frame_width": float(frame_width), "frame_height": float(frame_height)}
311
312     def detections_over_windows(self, video_path: str, windows: List[Tuple[int, int]],
313                                frame_skip: int = 3,
314                                progress_cb: Optional[Callable[[int, int], None]] = None
315                                ) -> List[Dict[str, Any]]:
316         """Per-window person detections via ONE sequential pass over the video.
317         Opens the video ONCE and reads forward ? ``grab()`` to advance, ``retrieve()`` +
318         detect only on sampled frames inside a window ? instead of

```

```

``detections_per_frame``'s
313         per-window open + ``POS_FRAMES`` seek, which on a long video degrades to
314         decode-from-start per window (0(window_index) ? the fusion-P3 extraction
bottleneck;
315         172 windows took ~80 min). ``windows`` = ``[(start_frame, end_frame), ...]``;
returns a
316         list aligned to it, each ``{"frames": {idx: [(tid,bbox)]}, "fps", "frame_width",
317         "frame_height"}``` (same shape as ``detections_per_frame``). Each window gets its
own
318         window-local ByteTrack. ``progress_cb(frames_done, total_frames)`` (called every
~256
319         frames) drives progress logging.
320         """
321         self.lazy_load_model()
322         cap = cv2.VideoCapture(video_path)
323         n = len(windows)
324         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
325         if fps <= 0:
326             fps = 30.0
327         fw = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
328         fh = cap.get(cv2.CAP_PROP_FRAME_HEIGHT) if cap.isOpened() else 0.0
329         base = {"fps": fps, "frame_width": float(fw), "frame_height": float(fh)}
330         if not cap.isOpened() or n == 0:
331             cap.release()
332             return [dict(frames={}, **base) for _ in range(n)]
333
334         step = max(frame_skip, 1)
335         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
336         frames_per_win: List[Dict[int, Any]] = [{ } for _ in range(n)]
337         trackers = [self.make_tracker(fps / step) for _ in range(n)]
338         max_end = max(ef for _, ef in windows)
339
340         frame_idx = 0
341         while frame_idx <= max_end:
342             if not cap.grab():
343                 break
344             # Windows that want THIS frame (inside the range AND on that window's
sampling grid).
345             wis = [wi for wi, (sf, ef) in enumerate(windows)
346                   if sf <= frame_idx <= ef and (frame_idx - sf) % step == 0]
347             if wis:
348                 ok, frame = cap.retrieve()
349                 if ok:
350                     for wi in wis:
351                         frames_per_win[wi][frame_idx] = self.detect_and_track(
352                             frame, trackers[wi], score_thresh)
353                 frame_idx += 1
354                 if progress_cb is not None and frame_idx % 256 == 0:
355                     progress_cb(frame_idx, max_end)
356             cap.release()
357             return [dict(frames=frames_per_win[wi], **base) for wi in range(n)]
358

```

10. assistant (2026-06-18T07:48:06.265Z)

Now let me read wasb_runner.py and the matching test files for the windowing logic.

11. user (2026-06-18T07:48:07.204Z)

```

1         """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3         Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,

```



```

6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21    TrackNetRunner.
22    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
24    wsl_tilde_quote
25
26    logger = logging.getLogger(__name__)
27
28    # Windows path to the inference wrapper we stage into WSL.
29    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
30
31    @dataclass
32    class WasbConfig:
33        """Resolved settings for a WASB WSL run (built from config.json ->
34        indexing.wasb)."""
35        distro: str = "Ubuntu"
36        repo_dir: str = "~/models/WASB-SBDT"
37        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
38        conda_env: str = "wasb"
39        weights_path: str = "~/models/WASB-
40        SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
41        sport: str = "badminton"
42        wsl_stage_dir: str = "~/clips"
43        timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
44        keep_frames: bool = False # retain staged video + frame cache after success (debug
45        only)
46        min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
47        off)
48
49        # FAST PATH (default OFF until verified): decode the staged video on the fly inside
50        # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
51        # extraction that dominates a long clip. Output is bit-identical to the PNG path
52        # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
53        it.
54
55        stream_video: bool = False
56
57    @classmethod
58    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
59        w = (idx_cfg or {}).get("wasb", {}) or {}
60        return cls(
61            distro=w.get("wsl_distro", "Ubuntu"),
62            repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
63            conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
64            conda_env=w.get("conda_env", "wasb"),
65            weights_path=w.get("weights_path",
66                               "~/models/WASB-
67                               SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
68            sport=w.get("sport", "badminton"),
69            wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
70            timeout_sec=int(w.get("timeout_sec", 1800)),
71            keep_frames=bool(w.get("keep_frames", False)),

```

```

63         min_free_gb=float(w.get("min_free_gb", 0.0)),
64         stream_video=bool(w.get("stream_video", False)),
65     )
66
67
68 class WasbRunner(WslCommandMixin, DetectorRunner):
69     def __init__(self, cfg: WasbConfig):
70         self.cfg = cfg
71
72     def healthcheck(self) -> Tuple[bool, str]:
73         """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74         cmd = (f"conda activate {self.cfg.conda_env} && "
75               f"python -c \"import torch; print('torch', torch.__version__, "
76               f"'cuda', torch.cuda.is_available())\"")
77         try:
78             res = self._wsl_bash(cmd)
79         except FileNotFoundError:
80             return False, "`wsl` not found ? Windows + WSL2 required."
81         except subprocess.TimeoutExpired:
82             return False, "WSL healthcheck timed out."
83         out = (res.stdout or "").strip()
84         if res.returncode != 0:
85             return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86         return True, out
87
88     def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89         src_mnt = to_wsl_mnt_path(video_win_path)
90         reason = self.wsl_source_unreadable_reason(src_mnt)
91         if reason:
92             logger.error("Cannot stage video into WSL: %s", reason)
93             return None
94         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} "
96         {wsl_tilde_quote(dest)} && echo OK"
97         try:
98             res = self._wsl_bash(cmd)
99         except subprocess.TimeoutExpired:
100             logger.error("Staging video into WSL timed out.")
101             return None
102         if res.returncode != 0 or "OK" not in (res.stdout or ""):
103             logger.error("Failed to stage video into WSL: %s", (res.stderr or
104             res.stdout).strip())
105             return None
106         return dest
107
108     def _stage_infer_script(self) -> bool:
109         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
110         src_mnt = to_wsl_mnt_path(_INFER_WIN)
111         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
112         OK"
113         res = self._wsl_bash(cmd)
114         return res.returncode == 0 and "OK" in (res.stdout or "")
115
116     def run_predict(self, video_win_path: str, output_win_dir: str,
117                    video_id: Optional[str] = None) -> Optional[str]:
118         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
119
120         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
121         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
122         can never reuse each other's staged frames, resumable cache, or CSV.
123
124         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
125         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
126         # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
127         # the CSV landed inside WSL while Windows polled the repo-relative path.

```

```

125     output_win_dir = os.path.abspath(output_win_dir)
126     os.makedirs(output_win_dir, exist_ok=True)
127     # Normalize for cross-platform basename (tests use Windows paths on Linux).
128     video_win_path = str(video_win_path).replace("\\", "/")
129     base = os.path.basename(video_win_path)
130     stem, ext = os.path.splitext(base)
131     # Unique, content-derived key: same filename + different bytes -> different key.
132     key = f"{stem}__{video_id[:12]}" if video_id else stem
133     wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134     expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136     if not self._stage_infer_script():
137         logger.error("Could not stage wasb_infer.py into WSL.")
138         return None
139     staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140     if staged_video is None:
141         return None
142     frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144     # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145     # WSL stage filesystem is already low so the user can clean up before it fills.
146     # (Streaming never materializes the PNGs, so the guard only matters off the fast
147     path.)
148     if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
149         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
150         if free is not None and free < self.cfg.min_free_gb:
151             logger.warning(
152                 "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
153                 extraction "
154                 "may fill the disk ? consider running tools/cleanup_caches.py.",
155                 free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
156
157     # Fast path: stream-decode the video in-process (no PNG extraction). Output is
158     # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
159     clip.
160     if self.cfg.stream_video:
161         frame_args = "--stream-video "
162     else:
163         frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
164     cmd = (
165         f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
166         f"python wasb_infer.py "
167         f"--video {wsl_tilde_quote(staged_video)} "
168         f"{frame_args}"
169         f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
170         f"--sport {shlex.quote(self.cfg.sport)} "
171         f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
172     )
173     logger.info("Running WASB inference on %s ...", base)
174     try:
175         res = self._wsl_bash(cmd)
176     except subprocess.TimeoutExpired:
177         logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
178         return None
179     if res.returncode != 0:
180         logger.error("WASB inference failed:\n%s", (res.stderr or
181         res.stdout)[-2000:])
182         return None
183     if not os.path.exists(expected_csv_win):
184         logger.error("WASB finished but expected CSV not found at %s",
185         expected_csv_win)
186         return None
187     logger.info("WASB trajectory CSV: %s", expected_csv_win)
188
189     # Success ? the trajectory CSV is the durable output; the staged video copy and

```

```

185         # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187         # so the frame cache + detector cache remain for a resumable re-run.
188         # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189         if not self.cfg.keep_frames:
190             stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191             logger.info("Cache hygiene: removing staged video%s for %s "
192                         "(set indexing.wasb.keep_frames=true to retain).",
193                         "" if self.cfg.stream_video else " + frame cache", key)
194             self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

12. user (2026-06-18T07:48:07.820Z)

```

1     """Tests for the TrackNet runner's pure-Python pieces (path translation, CSV
2     parsing, trajectory -> action-window conversion). The WSL subprocess invocation
3     itself is not exercised here ? these run anywhere, no WSL/TensorFlow needed."""
4
5     import os
6     import subprocess
7     import textwrap
8     from unittest.mock import patch
9
10    from backend.pipeline.detectors.tracknet_runner import (
11        TrackNetRunner,
12        TrackNetConfig,
13        TrajectoryPoint,
14        to_wsl_mnt_path,
15    )
16    from backend.pipeline.detectors.base import DetectorRunner
17    from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
18
19
20    def _proc(returncode=0, stdout="", stderr=""):
21        return subprocess.CompletedProcess(args=[], returncode=returncode,
22                                           stdout=stdout, stderr=stderr)
23
24
25    def test_to_wsl_mnt_path_windows():
26        assert to_wsl_mnt_path(r"C:\Users\avidu\v.mp4") == "/mnt/c/Users/avidu/v.mp4"
27        assert to_wsl_mnt_path(r"D:\clips\m.mp4") == "/mnt/d/clips/m.mp4"
28
29
30    def test_to_wsl_mnt_path_passthrough():
31        # Already-POSIX paths are left alone.
32        assert to_wsl_mnt_path("/home/avidullu/clips/v.mp4") == "/home/avidullu/clips/v.mp4"
33        assert to_wsl_mnt_path("~/clips/v.mp4") == "~/clips/v.mp4"
34
35
36    def test_config_from_indexing_cfg():
37        cfg = TrackNetConfig.from_indexing_cfg({"tracknet": {
38            "wsl_distro": "Ubuntu-22.04", "conda_env": "TN", "weights_path": "/w/m.h5",
39            "queue_length": 7, "stage_in_wsl": False,
40        }})
41        assert cfg.distro == "Ubuntu-22.04"
42        assert cfg.conda_env == "TN"

```

```

43     assert cfg.weights_path == "/w/m.h5"
44     assert cfg.queue_length == 7
45     assert cfg.stage_in_wsl is False
46
47
48     def test_config_defaults_when_absent():
49         cfg = TrackNetConfig.from_indexing_cfg({})
50         assert cfg.distro == "Ubuntu"
51         assert cfg.conda_env == "TrackNetV4"
52         assert cfg.weights_path == "" # must be supplied by the user
53
54
55     def test_parse_trajectory_csv(tmp_path):
56         csv_file = tmp_path / "v_predict.csv"
57         csv_file.write_text(textwrap.dedent("""\
58             Frame,Visibility,X,Y
59             0,0,0,0
60             1,1,100,200
61             2,1,140,210
62             3,0,0,0
63         """))
64         pts = TrackNetRunner.parse_trajectory_csv(str(csv_file))
65         assert len(pts) == 4
66         assert pts[0].visible is False
67         assert pts[1].visible is True and pts[1].x == 100 and pts[1].y == 200
68         assert pts[3].visible is False
69
70
71     def test_trajectory_to_action_windows_detects_moving_shuttle():
72         # 30 fps, 1280px wide. Shuttle moves ~60px/frame while visible (frames 0..29) =>
73         # well above the velocity threshold; then disappears. Expect one window ~1s long.
74         fps, width = 30.0, 1280.0
75         pts = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in
76 range(30)]
77
78         pts += [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(30, 60)]
79
80         windows = TrackNetRunner.trajectory_to_action_windows(
81             pts, fps=fps, frame_width=width, velocity_thresh=0.02,
82             merge_gap=2.0, min_window_duration=0.5, chunk_index=0,
83         )
84         assert len(windows) == 1
85         w = windows[0]
86         assert w["start"] < w["end"]
87         assert w["active_frame_count"] > 0
88         assert w["peak_velocity"] > 0.02
89         assert w["chunk_index"] == 0
90
91
92     def test_trajectory_to_action_windows_ignores_stationary_shuttle():
93         # Visible but barely moving (1px/frame) => below threshold => no windows.
94         pts = [TrajectoryPoint(frame=i, visible=True, x=500 + i, y=300) for i in range(60)]
95         windows = TrackNetRunner.trajectory_to_action_windows(
96             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.05,
97             merge_gap=2.0, min_window_duration=0.5,
98         )
99         assert windows == []
100
101
102     def test_trajectory_to_action_windows_splits_on_gap():
103         # Two bursts of motion separated by a long invisible gap => two windows.
104         burst1 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * i, y=300) for i in
105 range(20)]
106
107         gap = [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(20, 140)]
108         burst2 = [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * (i - 140), y=300)
109 for i in range(140, 160)]

```

```

106     windows = TrackNetRunner.trajectory_to_action_windows(
107         burst1 + gap + burst2, fps=30.0, frame_width=1280.0,
108         velocity_thresh=0.02, merge_gap=2.0, min_window_duration=0.3,
109     )
110     assert len(windows) == 2
111     assert windows[0]["end"] < windows[1]["start"]
112
113
114     # ---- W1: bounded windowing (max_window_duration split)
115     ----- #
116     def _moving(a, b): # visible, fast-moving shuttle over frames [a, b)
117         return [TrajectoryPoint(frame=i, visible=True, x=100 + 60 * (i - a), y=300) for i in
118             range(a, b)]
119
120     def _invisible(a, b):
121         return [TrajectoryPoint(frame=i, visible=False, x=0, y=0) for i in range(a, b)]
122
123     # Two rallies merged across a sub-merge_gap dead time ? ONE over-long window without the
124     cap.
125     _MERGED_PTS = _moving(0, 121) + _invisible(121, 162) + _moving(162, 243) # ~8s, 1.4s
126     internal gap
127
128     def test_max_window_duration_off_keeps_over_merge():
129         # Default (max_window_duration=0): the two rallies stay merged into one window.
130         w = TrackNetRunner.trajectory_to_action_windows(
131             _MERGED_PTS, fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
132             merge_gap=2.0, min_window_duration=0.3)
133         assert len(w) == 1 # over-merge reproduced (the failure the cap fixes)
134         assert (w[0]["end"] - w[0]["start"]) > 7.0
135
136     def test_max_window_duration_splits_at_inactivity_gap():
137         # With a 4s cap the 8s merged window is split at its 1.4s internal dead-time gap ? 2
138         windows.
139         w = TrackNetRunner.trajectory_to_action_windows(
140             _MERGED_PTS, fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
141             merge_gap=2.0, min_window_duration=0.3, max_window_duration=4.0,
142             min_split_gap=0.5)
143         assert len(w) == 2
144         assert w[0]["end"] < w[1]["start"]
145         assert all((x["end"] - x["start"]) <= 5.0 for x in w)
146
147     def test_max_window_duration_preserves_genuine_long_rally():
148         # A single continuous 10s rally (no internal gap ? min_split_gap) is NOT split, even
149         # though it exceeds the cap ? bounded windowing only cuts at real dead time.
150         w = TrackNetRunner.trajectory_to_action_windows(
151             _moving(0, 301), fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
152             merge_gap=2.0, min_window_duration=0.3, max_window_duration=4.0,
153             min_split_gap=0.5)
154         assert len(w) == 1
155         assert (w[0]["end"] - w[0]["start"]) > 9.0
156
157     def test_split_overlong_groups_pure():
158         # Direct check of the splitter: a group with one big internal gap splits there.
159         grp = [(i, 1.0) for i in range(0, 20)] + [(i, 1.0) for i in range(60, 80)] # gap
160         20..60
161         out = TrackNetRunner._split_overlong_groups([grp], max_win_frames=30,
162             min_split_frames=10)
163         assert len(out) == 2 and out[0][-1][0] == 19 and out[1][0][0] == 60
164         # No qualifying gap (all consecutive) ? unchanged even when over length.

```

```

162     solid = [(i, 1.0) for i in range(0, 100)]
163     assert TrackNetRunner._split_overlong_groups([solid], 30, 10) == [solid]
164
165
166     # ---- W1 hard-cap fallback: enforce the cap on continuously-active trajectories
167     ----- #
168     def test_window_hard_cap_chops_continuous_rally():
169         # The same continuous ~10s rally that bounded-windowing alone LEAVES intact (no
170         internal
171         # gap) is, with window_hard_cap=True, chopped at the midpoint until every piece ?
172         the cap.
173         w = TrackNetRunner.trajectory_to_action_windows(
174             _moving(0, 301), fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
175             merge_gap=2.0, min_window_duration=0.3, max_window_duration=4.0,
176             min_split_gap=0.5,
177             window_hard_cap=True)
178         assert len(w) >= 3 # the 10s blob is broken
179         up
180         assert all((x["end"] - x["start"]) <= 4.01 for x in w) # every piece within the
181         cap
182         assert w[0]["start"] <= 0.1 and w[-1]["end"] >= 9.0 # coverage preserved
183         (only cuts)
184
185     def test_split_overlong_groups_hard_cap_pure():
186         solid = [(i, 1.0) for i in range(0, 100)] # 99-frame span, no
187         internal gap
188         # default: kept whole; hard_cap: chopped into pieces each spanning ? max_win_frames
189         assert TrackNetRunner._split_overlong_groups([solid], 30, 10) == [solid]
190         out = TrackNetRunner._split_overlong_groups([solid], 30, 10, hard_cap=True)
191         assert len(out) >= 4
192         assert all((g[-1][0] - g[0][0]) <= 30 for g in out)
193         # only cuts, never drops/duplicates: the pieces partition the original frames
194         exactly
195         assert sorted(f for g in out for f, _ in g) == list(range(0, 100))
196
197     # ---- R4: rally-END inactivity trim (drop the post-rally slow-drift tail)
198     ----- #
199     def test_trim_inactive_tail_drops_slow_drift():
200         # fast rally (vel 0.5) frames 0..60, then slow drift (vel 0.03, still > active 0.02)
201         61..200.
202         group = [(i, 0.5) for i in range(0, 61)] + [(i, 0.03) for i in range(61, 201)]
203         out = TrackNetRunner._trim_inactive_tail(group, trail_frames=30, min_density=0.3,
204             rally_thresh=0.2)
205         assert out[0][0] == 0 # start untouched
206         assert out[-1][0] < 100 # the pure-drift tail (frames
207         100..200) is trimmed off
208         assert len(group) - len(out) >= 100 # ~all the slow drift cut (kept only a
209         short decay tail)
210
211     def test_trim_inactive_tail_keeps_continuous_rally_and_is_noop_when_off():
212         allfast = [(i, 0.5) for i in range(0, 100)]
213         assert TrackNetRunner._trim_inactive_tail(allfast, 30, 0.3, 0.2) == allfast #
214         fully-fast: untouched
215         assert TrackNetRunner._trim_inactive_tail(allfast, 0, 0.3, 0.2) == allfast #
216         trail=0: OFF/no-op
217
218     def test_window_inactivity_end_trims_through_api():
219         # one window of fast motion then slow drift (both visible & > velocity_thresh ? one
220         group).
221         fast = [TrajectoryPoint(frame=i, visible=True, x=100 + 40 * (i % 5), y=300) for i in
222             range(0, 90)]

```

```

209         slow = [TrajectoryPoint(frame=i, visible=True, x=100 + (i % 2), y=300) for i in
range(90, 300)]
210         pts = fast + slow
211         off = TrackNetRunner.trajectory_to_action_windows(
212             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=0.3)
213         on = TrackNetRunner.trajectory_to_action_windows(
214             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
min_window_duration=0.3,
215             window_inactivity_end=1.0, inactivity_rally_thresh=0.2,
inactivity_min_density=0.3)
216         assert on and off
217         assert on[0]["end"] < off[0]["end"]          # R4 trims the drift tail ? earlier END
218         assert on[0]["start"] == off[0]["start"]      # START unchanged
219
220
221     # ---- R5: blob-fallback (force-split the continuously-active blob, default OFF)
----- #
222     def test_blob_fallback_chop_pure():
223         blob = [(i, 1.0) for i in range(0, 200)]      # 199-frame span, no gap = 6.6× a
30-frame cap
224         out, flags = TrackNetRunner._blob_fallback_chop([blob], max_win_frames=30,
blob_factor=4.0)
225         assert len(out) >= 4 and all(flags)           # force-chopped, every
piece flagged
226         assert all((g[-1][0] - g[0][0]) <= 30 for g in out)      # every piece within
the cap
227         assert sorted(f for g in out for f, _ in g) == list(range(0, 200)) # partition
(only cuts)
228
229
230     def test_blob_fallback_factor_leaves_normal_long_rally_alone():
231         # 79-frame span = 2.6× a 30-frame cap ? BELOW the 4× blob threshold ? left ALONE
(the gate that
232         # keeps R5 surgical; this is the over-cap-but-not-a-blob case hard_cap would wrongly
chop).
233         normal = [(i, 1.0) for i in range(0, 80)]
234         out, flags = TrackNetRunner._blob_fallback_chop([normal], max_win_frames=30,
blob_factor=4.0)
235         assert out == [normal] and flags == [False]
236
237
238     def test_window_blob_fallback_rescues_blob_through_api():
239         # A continuous ~10s blob (no gap) the cap alone leaves whole: blob_fallback breaks
it up + flags.
240         pts = _moving(0, 301)                          # 10s continuous, no
internal gap
241         off = TrackNetRunner.trajectory_to_action_windows(
242             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
243             min_window_duration=0.3, max_window_duration=2.0)      # cap 2s, blob_fallback
OFF
244         on = TrackNetRunner.trajectory_to_action_windows(
245             pts, fps=30.0, frame_width=1280.0, velocity_thresh=0.02, merge_gap=2.0,
246             min_window_duration=0.3, max_window_duration=2.0,
247             window_blob_fallback=True, window_blob_factor=4.0)      # 10s > 4×2s threshold ?
a blob
248         assert len(off) == 1 and (off[0]["end"] - off[0]["start"]) > 9.0    # cap alone
leaves it whole
249         assert len(on) >= 4          # the blob is broken up
250         assert all((w["end"] - w["start"]) <= 2.01 for w in on)      # every piece within the
cap
251         assert all(w.get("forced_cut") for w in on)          # every piece flagged a
forced cut
252
253

```



```

254     def test_window_blob_fallback_noop_without_cap():
255         # blob_fallback requires a cap to define "over-cap"; with max_window_duration=0 it's
a no-op.
256         w = TrackNetRunner.trajectory_to_action_windows(
257             _moving(0, 301), fps=30.0, frame_width=1280.0, velocity_thresh=0.02,
merge_gap=2.0,
258             min_window_duration=0.3, window_blob_fallback=True)
259         assert len(w) == 1 and not w[0].get("forced_cut")
260
261
262     # ----- WSL subprocess glue
263
264     def _runner():
265         return TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5"))
266
267
268     def test_healthcheck_ok():
269         r = _runner()
270         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="TF 2.17.0 GPUs 1")):
271             ok, msg = r.healthcheck()
272             assert ok is True
273             assert "TF 2.17.0" in msg
274
275
276     def test_healthcheck_failure_returns_message():
277         r = _runner()
278         with patch.object(r, "_wsl_bash", return_value=_proc(1,
stderr="ModuleNotFoundError")):
279             ok, msg = r.healthcheck()
280             assert ok is False
281             assert "ModuleNotFoundError" in msg
282
283
284     def test_healthcheck_no_wsl_binary():
285         r = _runner()
286         with patch.object(r, "_wsl_bash", side_effect=FileNotFoundError()):
287             ok, msg = r.healthcheck()
288             assert ok is False
289             assert "wsl" in msg.lower()
290
291
292     def test_run_predict_requires_weights():
293         r = TrackNetRunner(TrackNetConfig(weights_path="")) # no weights configured
294         assert r.run_predict(r"C:\v\clip.mp4", "out") is None
295
296
297     def test_run_predict_rejects_bad_extension(tmp_path):
298         r = _runner()
299         assert r.run_predict(r"C:\v\clip.mkv", str(tmp_path)) is None
300
301
302     def test_run_predict_success(tmp_path):
303         r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
304         out_dir = str(tmp_path)
305         # predict.py would write "<stem>_predict.csv"; create it so the existence check
passes.
306         csv_path = os.path.join(out_dir, "clip_predict.csv")
307         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="Prediction
complete.")):
308             open(csv_path, "w").close()
309             result = r.run_predict(r"C:\v\clip.mp4", out_dir)
310             assert result == csv_path
311
312
313     def test_run_predict_failure_returns_none(tmp_path):

```

```

314         r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
315         with patch.object(r, "_wsl_bash", return_value=_proc(1, stderr="CUDA error")):
316             assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None
317
318
319     def test_run_predict_missing_output_csv(tmp_path):
320         """Exit 0 but no CSV produced should be treated as failure."""
321         r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=False))
322         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
323             assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path)) is None
324
325
326     def test_stage_video_success():
327         r = _runner()
328         # Two WSL calls now: the readability preflight (echo READABLE), then the cp (echo
329         OK).
330         with patch.object(r, "_wsl_bash",
331                             side_effect=[_proc(0, stdout="READABLE"), _proc(0, stdout="OK")]):
332             staged = r._stage_video(r"C:\v\clip.mp4", "clip.mp4")
333             assert staged == "~/clips/clip.mp4"
334
335     def test_stage_video_failure():
336         r = _runner()
337         # Source readable, but the cp itself fails -> None.
338         with patch.object(r, "_wsl_bash",
339                             side_effect=[_proc(0, stdout="READABLE"), _proc(1, stderr="disk
340         full")]):
341             assert r._stage_video(r"C:\v\clip.mp4", "clip.mp4") is None
342
343     def test_stage_video_unreadable_source_fails_fast_with_actionable_msg(caplog):
344         """A source invisible to WSL (Google Drive / UNC) must fail BEFORE cp, with a clear,
345         fixable message ? not a cryptic 'cannot stat' that silently yields 0 rallies."""
346         r = _runner()
347         import logging
348         # Preflight `test -r` returns no READABLE -> unreadable. cp must NOT be attempted.
349         with patch.object(r, "_wsl_bash", return_value=_proc(1, stdout="")) as wsl, \
350             caplog.at_level(logging.ERROR):
351             assert r._stage_video("G:\My Drive\Sharing\clip.mp4", "clip.mp4") is None
352             assert wsl.call_count == 1 # only the preflight ran; cp short-circuited
353             assert "not readable from WSL" in caplog.text
354             assert "local disk" in caplog.text
355
356
357     def test_wsl_source_unreadable_reason_none_when_readable():
358         r = _runner()
359         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="READABLE")):
360             assert r.wsl_source_unreadable_reason("/mnt/c/v/clip.mp4") is None
361
362
363     def test_wsl_source_unreadable_reason_none_on_flaky_probe():
364         """A timed-out/raising probe must NOT block a genuinely-readable source."""
365         r = _runner()
366         with patch.object(r, "_wsl_bash", side_effect=subprocess.TimeoutExpired(cmd="x",
367         timeout=1)):
368             assert r.wsl_source_unreadable_reason("/mnt/c/v/clip.mp4") is None
369
370     def test_run_predict_stages_when_enabled(tmp_path):
371         """With stage_in_wsl=True, run_predict should stage the video first."""
372         r = TrackNetRunner(TrackNetConfig(weights_path="/w/model.h5", stage_in_wsl=True))
373         out_dir = str(tmp_path)
374         csv_path = os.path.join(out_dir, "clip_predict.csv")
375         with patch.object(r, "_stage_video", return_value="~/clips/clip.mp4") as mock_stage,

```

```

\
376         patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")):
377         open(csv_path, "w").close()
378         result = r.run_predict(r"C:\v\clip.mp4", out_dir)
379     mock_stage.assert_called_once()
380     assert result == csv_path
381
382
383     # ----- DetectorRunner ABC
contract
384
385     def test_tracknet_runner_implements_detector_runner_abc():
386         """TrackNetRunner must be a DetectorRunner (for polymorphism / future native
runners)."""
387         r = _runner()
388         assert isinstance(r, DetectorRunner)
389         # healthcheck contract: (bool, str) ? exercised with patch in other tests, but shape
here too
390         with patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="TF ok")):
391             res = r.healthcheck()
392             assert isinstance(res, tuple) and len(res) == 2
393             assert isinstance(res[0], bool)
394             assert isinstance(res[1], str)
395
396
397     def test_wasb_runner_implements_detector_runner_abc():
398         """WasbRunner must also satisfy the DetectorRunner contract."""
399         w = WasbRunner(WasbConfig())
400         assert isinstance(w, DetectorRunner)
401         with patch.object(w, "_wsl_bash", return_value=_proc(0, stdout="torch ok")):
402             res = w.healthcheck()
403             assert isinstance(res, tuple) and len(res) == 2
404             assert isinstance(res[0], bool) and isinstance(res[1], str)
405
406
407     # ----- WASB streaming fast-path wiring
408
409     def test_wasb_config_stream_video_default_off_and_pares():
410         """Streaming is OFF by default (production behavior unchanged) and opt-in via
cfg."""
411         assert WasbConfig().stream_video is False
412         cfg = WasbConfig.from_indexing_cfg({"wasb": {"stream_video": True}})
413         assert cfg.stream_video is True
414         assert WasbConfig.from_indexing_cfg({"wasb": {}}).stream_video is False
415
416
417     def _wasb_run_predict_cmd(stream_video, tmp_path):
418         """Drive WasbRunner.run_predict with WSL fully mocked and return the inference cmd
419         string (the last _wsl_bash call carries `python wasb_infer.py ...`)."""
420         r = WasbRunner(WasbConfig(stream_video=stream_video))
421         out_dir = str(tmp_path)
422         key = "clip__abc123abc123"
423         csv_path = os.path.join(out_dir, f"{key}_wasb.csv")
424         with patch.object(r, "_stage_infer_script", return_value=True), \
425             patch.object(r, "_stage_video", return_value=~/.clips/clip.mp4"), \
426             patch.object(r, "_wsl_rm_rf"), \
427             patch.object(r, "_wsl_bash", return_value=_proc(0, stdout="done")) as wsl:
428             open(csv_path, "w").close()
429             result = r.run_predict(r"C:\v\clip.mp4", out_dir, video_id="abc123abc123def")
430             assert result == csv_path
431             # The inference invocation is the call whose command contains wasb_infer.py.
432             infer_cmds = [c.args[0] for c in wsl.call_args_list if "wasb_infer.py" in c.args[0]]
433             assert len(infer_cmds) == 1
434             return infer_cmds[0]
435

```

```

436
437 def test_run_predict_streaming_passes_stream_flag_not_frames_dir(tmp_path):
438     cmd = _wasb_run_predict_cmd(stream_video=True, tmp_path=tmp_path)
439     assert "--stream-video" in cmd
440     assert "--frames_out_dir" not in cmd    # no PNG extraction on the fast path
441     assert "--video " in cmd                # still decodes the staged video
442
443
444 def test_run_predict_default_passes_frames_dir_not_stream_flag(tmp_path):
445     cmd = _wasb_run_predict_cmd(stream_video=False, tmp_path=tmp_path)
446     assert "--stream-video" not in cmd      # default path unchanged
447     assert "--frames_out_dir" in cmd
448
449
450 # (Intentionally no "no-wsl unpatched predict" test here: the stage/inference paths
451 # raise FileNotFoundError on this Linux box when "wsl" is absent, while only the
452 # healthcheck paths catch it today. The graceful-None cases are already exercised
453 # by the patched tests above and by the hybrid tests. Adding a broad except in
454 # run_predict would be a separate robustness change.)
455

```

13. assistant (2026-06-18T07:48:10.841Z)

Now let me look closely at potential bugs. Let me examine the `\_trim\_inactive\_tail` carefully, and the `detections\_over\_windows` flow. Let me also check the geometry helper and the fusion tests.

14. user (2026-06-18T07:48:12.276Z)

```

1     """Unit tests for the person-detection rally cue
  (backend.pipeline.detectors.person_detector).
2
3     Pure/offline ? the torchvision model and supervision tracker are mocked, so these run
4     without GPU/torch weights. They pin the COCO person==1 filter (the YOLO-was-0 gotcha)
  and
5     the motion-window extraction that feeds the candidate `stats` + the learned fusion
  features.
6     """
7     from unittest.mock import MagicMock, patch
8
9     from backend.pipeline.detectors.person_detector import PersonDetector, _PERSON_CLASS_ID
10
11
12     def _make_cap_mock(num_frames=300):
13         """A cv2.VideoCapture mock yielding `num_frames` readable frames at 30fps."""
14         import cv2
15         cap_mock = MagicMock()
16         cap_mock.isOpened.return_value = True
17
18         def mock_get_prop(prop):
19             if prop == cv2.CAP_PROP_FPS:
20                 return 30.0
21             if prop == cv2.CAP_PROP_FRAME_WIDTH:
22                 return 1000.0
23             return 0.0
24
25         cap_mock.get.side_effect = mock_get_prop
26         cap_mock.read.side_effect = [(True, "frame")] * num_frames + [(False, None)]
27         return cap_mock
28
29
30     def _make_detections(num_frames=300, step=5, track_id=1):
31         """One steadily-moving person per frame (box advances `step` px/frame)."""
32         return [(track_id, (float(i * step), 0.0, float(i * step + 10), 10.0))]
33         for i in range(num_frames)]
34

```

```

34
35
36 @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
37 def test_extract_action_windows_returns_enriched_dicts(mock_cap):
38     pd = PersonDetector({"indexing": {"min_action_window_duration": 1.0}})
39     pd.model = MagicMock() # short-circuits
40     lazy_load_model
41     pd.make_tracker = MagicMock(return_value=MagicMock())
42     pd.detect_and_track = MagicMock(side_effect=_make_detections())
43     mock_cap.return_value = _make_cap_mock()
44
45     windows = pd.extract_action_windows_from_chunk(
46         "chunk.mp4", chunk_start=5.0, velocity_thresh=0.05, merge_gap=2.0,
47         frame_skip=1, chunk_index=2,
48     )
49
50     assert len(windows) >= 1
51     w = windows[0]
52     assert set(w) >= {"start", "end", "active_frame_count", "peak_velocity",
53                     "mean_velocity", "track_id_count", "chunk_index"}
54     assert w["chunk_index"] == 2
55     assert w["start"] >= 5.0 # full-video coordinates
56     assert w["active_frame_count"] > 0
57     assert w["peak_velocity"] >= w["mean_velocity"] > 0
58     assert w["track_id_count"] == 1
59
60 @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
61 def test_detections_per_frame(mock_cap):
62     """The per-frame path the fusion segmenter uses: samples [start,end] by frame_skip,
63     returns {frame_idx: dets} keyed by absolute video frame + the frame dims (incl.
64     height)."""
65     import cv2
66
67     pd = PersonDetector({"indexing": {"detector_score_threshold": 0.5}})
68     pd.model = MagicMock() # short-circuits
69     lazy_load_model
70     pd.make_tracker = MagicMock(return_value=MagicMock())
71     pd.detect_and_track = MagicMock(return_value=[(1, (10.0, 10.0, 30.0, 40.0))])
72
73     cap = _make_cap_mock(num_frames=20)
74
75     def _get(prop):
76         if prop == cv2.CAP_PROP_FPS:
77             return 30.0
78         if prop == cv2.CAP_PROP_FRAME_WIDTH:
79             return 1000.0
80         if prop == cv2.CAP_PROP_FRAME_HEIGHT:
81             return 800.0
82         return 0.0
83
84     cap.get.side_effect = _get
85     mock_cap.return_value = cap
86
87     out = pd.detections_per_frame("v.mp4", start_frame=0, end_frame=9, frame_skip=3)
88     assert out["fps"] == 30.0
89     assert out["frame_width"] == 1000.0 and out["frame_height"] == 800.0 # height
90     returned (the gap)
91     assert sorted(out["frames"].keys()) == [0, 3, 6, 9] # sampled every 3rd frame in
92     [0,9]
93     assert out["frames"][0] == [(1, (10.0, 10.0, 30.0, 40.0))]
94     cap.release.assert_called_once()
95
96 @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")

```

```

94     def test_detections_over_windows_single_pass(mock_cap):
95         """One sequential pass dispatches sampled frames to the right windows (no per-window
seek)."""
96         import cv2
97
98         pd = PersonDetector({"indexing": {"detector_score_threshold": 0.5}})
99         pd.model = MagicMock()
100         pd.make_tracker = MagicMock(return_value=MagicMock())
101         pd.detect_and_track = MagicMock(return_value=[(1, (10.0, 10.0, 30.0, 40.0))])
102
103         cap = MagicMock()
104         cap.isOpened.return_value = True
105         cap.get.side_effect = lambda p: {cv2.CAP_PROP_FPS: 30.0, cv2.CAP_PROP_FRAME_WIDTH:
1000.0,
106                                         cv2.CAP_PROP_FRAME_HEIGHT: 800.0}.get(p, 0.0)
107         cap.grab.side_effect = [True] * 11 + [False] # frames 0..10 readable
108         cap.retrieve.return_value = (True, "frame")
109         mock_cap.return_value = cap
110
111         windows = [(0, 6), (8, 10)] # frame_skip=2
112         out = pd.detections_over_windows("v.mp4", windows, frame_skip=2)
113
114         assert len(out) == 2
115         assert out[0]["frame_width"] == 1000.0 and out[0]["frame_height"] == 800.0
116         assert sorted(out[0]["frames"].keys()) == [0, 2, 4, 6] # window 0 on its sampling
grid
117         assert sorted(out[1]["frames"].keys()) == [8, 10] # window 1
118         mock_cap.assert_called_once() # opened ONCE for all
windows
119
120
121     def test_detect_and_track_filters_persons_and_returns_track_ids():
122         """detect_and_track keeps only confident PERSON detections, feeds them to the
123         tracker, and returns (track_id, bbox) tuples ? exercising the real torchvision
124         output-shape handling and the COCO person==1 filter (the YOLO-was-0 gotcha)."""
125         import numpy as np
126
127         pd = PersonDetector({"indexing": {}})
128         pd.device = "cpu"
129
130         # Fake torchvision detector output: a person (kept), a low-score person (dropped),
131         # and a racket/other class (dropped).
132         out = {"boxes": MagicMock(), "labels": MagicMock(), "scores": MagicMock()}
133         out["boxes"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
134             [[0, 0, 10, 10], [5, 5, 15, 15], [20, 20, 30, 30]], dtype=float)
135         out["labels"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
136             [_PERSON_CLASS_ID, _PERSON_CLASS_ID, 38]) # 38 = sports-ball-ish, not
person
137         out["scores"].detach.return_value.cpu.return_value.numpy.return_value = np.array(
138             [0.9, 0.1, 0.99])
139         pd.model = MagicMock(return_value=out)
140
141         # Fake tracker echoes the detections it receives, assigning a single track id.
142         tracker = MagicMock()
143         tracked = MagicMock()
144         tracked.__len__.return_value = 1
145         tracked.tracker_id = [7]
146         tracked.xyxy = np.array([[0, 0, 10, 10]], dtype=float)
147         tracker.update_with_detections.return_value = tracked
148
149         with patch("backend.pipeline.detectors.person_detector.cv2.cvtColor",
return_value=np.zeros((10, 10, 3))), \
150             patch("backend.pipeline.detectors.person_detector.torch") as mock_torch, \
151             patch.dict("sys.modules", {"supervision": MagicMock()}):
152             mock_torch.no_grad.return_value.__enter__ = lambda s: None

```

```

153         mock_torch.no_grad.return_value.__exit__ = lambda s, *a: None
154         # torch.from_numpy(...).permute(...).float().div(...).to(...) -> dummy tensor
155         mock_torch.from_numpy.return_value.permute.return_value.float.return_value.div.r
return_value.to.return_value = "tensor"
156         result = pd.detect_and_track("frame", tracker, score_thresh=0.5)
157
158         # Only the high-confidence person survived the score+class filter
159         assert result == [(7, (0.0, 0.0, 10.0, 10.0))]
160         # And exactly that filtered set (1 box) was handed to the tracker
161         tracker.update_with_detections.assert_called_once()
162

```

15. user (2026-06-18T07:48:12.562Z)

```

1         """Pure-logic unit tests for the person-cue fusion features
(backend.pipeline.detectors.fusion_features).
2
3         No GPU/torch/cv2 ? synthetic person boxes + shuttle points exercise every feature path
and
4         edge case. This is the testable core that proves the fusion math correct before any GPU
run.
5         """
6         from types import SimpleNamespace
7
8         import pytest
9
10        from backend.pipeline.detectors import fusion_features as ff
11
12
13        def _shuttle(frame, x, y, visible=True):
14            return SimpleNamespace(frame=frame, x=x, y=y, visible=visible)
15
16
17        # Square unit polygon and a left-half polygon (x in [0, 0.5]).
18        FULL = [(0.0, 0.0), (1.0, 0.0), (1.0, 1.0), (0.0, 1.0)]
19        LEFT_HALF = [(0.0, 0.0), (0.5, 0.0), (0.5, 1.0), (0.0, 1.0)]
20
21
22        # ----- point_in_polygon
23
24        def test_point_in_polygon_inside_outside():
25            assert ff.point_in_polygon((0.5, 0.5), FULL) is True
26            assert ff.point_in_polygon((1.5, 0.5), FULL) is False
27            assert ff.point_in_polygon((0.2, 0.5), LEFT_HALF) is True
28            assert ff.point_in_polygon((0.8, 0.5), LEFT_HALF) is False
29
30
31        def test_point_in_polygon_degenerate():
32            assert ff.point_in_polygon((0.5, 0.5), None) is False
33            assert ff.point_in_polygon((0.5, 0.5), [(0.0, 0.0), (1.0, 1.0)]) is False # <3
verts
34
35
36        # ----- person_in_court
37
38        def test_person_in_court_uses_foot_point_and_polygon():
39            # box foot = ((x1+x2)/2/W, y2/H). W=H=100.
40            in_box = (10, 10, 30, 30) # foot (0.2, 0.3) ? left half
41            out_box = (60, 10, 80, 30) # foot (0.7, 0.3) ? right half
42            assert ff.person_in_court(in_box, 100, 100, LEFT_HALF) is True
43            assert ff.person_in_court(out_box, 100, 100, LEFT_HALF) is False
44            # None polygon => everyone counts
45            assert ff.person_in_court(out_box, 100, 100, None) is True
46            # Unusable frame dims => foot point is meaningless; a masked check returns False
(explicit

```

```

47         # contract, not whatever the (0,0) ray-cast happens to give). No mask still counts
everyone.
48         assert ff.person_in_court(in_box, 0, 0, LEFT_HALF) is False
49         assert ff.person_in_court(in_box, 0, 0, None) is True
50
51
52     # ----- counts / ratios
53
54     def test_in_court_counts_and_players_and_ratio():
55         near = (10, 10, 30, 30)      # foot (0.2,0.3) in LEFT_HALF
56         far = (60, 10, 80, 30)      # foot (0.7,0.3) NOT in LEFT_HALF
57         per_frame = {0: [(1, near), (2, far)], 1: [(1, near)], 2: [], 3: [(1, near), (2,
near)]}]
58         # With LEFT_HALF mask: counts = [1, 1, 0, 2]
59         counts = ff.in_court_counts(per_frame, 100, 100, LEFT_HALF)
60         assert counts == [1, 1, 0, 2]
61         assert ff.players_in_court(counts) == 1.0          # median of [0,1,1,2]
62         assert ff.in_court_ratio(counts, min_players=2) == pytest.approx(0.25)
63         assert ff.in_court_ratio(counts, min_players=1) == pytest.approx(0.75)
64         # No mask: far counts too -> [2,1,0,2]
65         assert ff.in_court_counts(per_frame, 100, 100, None) == [2, 1, 0, 2]
66         assert ff.players_in_court([]) == 0.0
67         assert ff.in_court_ratio([]) == 0.0
68
69
70     # ----- mean_player_motion
71
72     def test_mean_player_motion():
73         # track 1 moves 10px in x between frames; W=H=100 -> normalized disp 0.10.
74         per_frame = {0: [(1, (0, 0, 10, 10))], 1: [(1, (10, 0, 20, 10))]}
75         assert ff.mean_player_motion(per_frame, 100, 100) == pytest.approx(0.10)
76         # single frame -> 0; no shared track id -> 0
77         assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))]}, 100, 100) == 0.0
78         assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))], 1: [(2, (10, 0, 20, 10))]},
100, 100) == 0.0
79
80
81     # ----- two_sided_motion
82
83     def test_two_sided_motion():
84         net = ("y", 0.5)
85         near = (10, 10, 30, 40)      # foot y = 0.4 < 0.5 (near side)
86         far = (10, 60, 30, 90)      # foot y = 0.9 >= 0.5 (far side)
87         both = {0: [(1, near), (2, far)], 1: [(1, near)]}      # frame 0 two-sided, frame 1
one-sided
88         assert ff.two_sided_motion(both, 100, 100, net, None) == pytest.approx(0.5)
89         one_sided = {0: [(1, near)], 1: [(2, near)]}
90         assert ff.two_sided_motion(one_sided, 100, 100, net, None) == 0.0
91         assert ff.two_sided_motion(both, 100, 100, None, None) == 0.0    # no net -> 0
92
93
94     def test_two_sided_motion_respects_court_polygon():
95         net = ("y", 0.5)
96         near = (10, 10, 30, 40)      # foot (0.2, 0.4) ? near side, inside LEFT_HALF
97         far_out = (60, 60, 80, 90)  # foot (0.7, 0.9) ? far side BUT outside LEFT_HALF
98         per_frame = {0: [(1, near), (2, far_out)]}
99         assert ff.two_sided_motion(per_frame, 100, 100, net, None) == 1.0    # both sides
present
100         # the polygon excludes the far-side person -> only the near side counts -> not two-
sided
101         assert ff.two_sided_motion(per_frame, 100, 100, net, LEFT_HALF) == 0.0
102
103
104     def test_shuttle_player_colocation_pad_halo():
105         per_frame = {0: [(1, (40, 40, 60, 60))]}      # raw box x in [40,60]; default pad

```



```

0.03*100=3px halo
106         assert ff.shuttle_player_colocation(per_frame, [_shuttle(0, 62, 50, True)], 100,
107         100) == 1.0 # in halo
108         assert ff.shuttle_player_colocation(per_frame, [_shuttle(0, 64, 50, True)], 100,
109         100) == 0.0 # past halo
110         # pad_frac=0 disables the halo -> 62 is outside the raw box
111         assert ff.shuttle_player_colocation(per_frame, [_shuttle(0, 62, 50, True)], 100,
112         100, pad_frac=0.0) == 0.0
113
114     # ----- shuttle_player_colocation
115
116     def test_shuttle_player_colocation():
117         box = (40, 40, 60, 60)
118         per_frame = {0: [(1, box)], 1: [(1, box)], 2: [(1, box)]}
119         # frame0 shuttle inside box (held), frame1 shuttle far (in-flight), frame2
120         invisible.
121         shuttle = [_shuttle(0, 50, 50, True), _shuttle(1, 95, 95, True), _shuttle(2, 50, 50,
122         False)]
123         # aligned visible frames = {0, 1}; colocated = {0} -> 0.5
124         assert ff.shuttle_player_colocation(per_frame, shuttle, 100, 100) ==
125         pytest.approx(0.5)
126         # all-held -> 1.0
127         held = [_shuttle(0, 50, 50, True), _shuttle(1, 50, 50, True)]
128         assert ff.shuttle_player_colocation(per_frame, held, 100, 100) == pytest.approx(1.0)
129         # no shuttle -> 0.0; shuttle frame not person-sampled -> skipped (no aligned -> 0.0)
130         assert ff.shuttle_player_colocation(per_frame, [], 100, 100) == 0.0
131         assert ff.shuttle_player_colocation(per_frame, [_shuttle(99, 50, 50, True)], 100,
132         100) == 0.0
133
134     # ----- aggregator
135
136     def test_compute_person_features_keys():
137         near = (10, 10, 30, 40)
138         far = (10, 60, 30, 90)
139         per_frame = {0: [(1, near), (2, far)], 1: [(1, near), (2, far)]}
140         shuttle = [_shuttle(0, 20, 25, True)]
141         feats = ff.compute_person_features(per_frame, shuttle, 100, 100,
142         court_polygon=None, net="y", 0.5))
143         assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
144         assert all(isinstance(v, float) for v in feats.values())
145         assert feats["two_sided_motion"] == pytest.approx(1.0) # both sides occupied every
146         frame
147
148     def test_compute_person_features_empty():
149         feats = ff.compute_person_features({}, [], 100, 100)
150         assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
151         assert all(v == 0.0 for v in feats.values())
152
153

```

16. assistant (2026-06-18T07:48:19.563Z)

Let me focus carefully on the `\_trim\_inactive\_tail` two-pointer logic and the `\_two\_sided\_motion`/`detections\_over\_windows` functions. Let me trace `\_two\_sided\_motion` ? when `net` is provided but `per\_frame` has no in-court persons, it divides by `len(per\_frame)`. That's fine. Let me look at the `detections\_over\_windows` `max\_end` / `wis` dispatch when windows overlap and the `\_index\_from\_synthetic\_path` edge.

Let me re-examine the `\_trim\_inactive\_tail` two-pointer window bounds. The condition `while frames[lo] <= frames[i] - trail\_frames` ? this excludes frames at exactly `frames[i] - trail\_frames`. Let me check the colocation `\_shuttle\_in\_box` uses `p.x`, `p.y` not `int(p.frame)` indexing issues.

Let me check the geometry helper used by person_detector and look at the `two\_sided\_motion` net boundary semantics versus tests more carefully, plus check `point\_in\_polygon` division edge.

17. user (2026-06-18T07:48:20.501Z)

```
1     import math
2     from typing import Tuple
3
4     def get_center(bbox: Tuple[float, float, float, float]) -> Tuple[float, float]:
5         """Calculate the center (x, y) of a bounding box.
6         bbox format: (x1, y1, x2, y2)
7         """
8         x1, y1, x2, y2 = bbox
9         return ((x1 + x2) / 2.0, (y1 + y2) / 2.0)
10
11    def get_distance(p1: Tuple[float, float], p2: Tuple[float, float]) -> float:
12        """Calculate the Euclidean distance between two points (x, y)."""
13        return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)
14
15    def get_velocity(bbox1: Tuple[float, float, float, float], bbox2: Tuple[float, float,
float, float], fps: float) -> float:
16        """Calculate velocity (pixels per second) of a bounding box moving between two
consecutive frames.
17        """
18        c1 = get_center(bbox1)
19        c2 = get_center(bbox2)
20        dist = get_distance(c1, c2)
21        return dist * fps
22
23    def get_velocity_normalised(bbox1: Tuple[float, float, float, float], bbox2:
Tuple[float, float, float, float],
24                               fps: float, frame_width: float) -> float:
25        """Calculate velocity as a fraction of frame width per second.
26
27        This makes the threshold resolution-independent:
28        - 0.15 means the center moved 15% of the frame width in one second.
29        - Works identically on 720p, 1080p, and 4K footage.
30
31        Args:
32            bbox1: Previous bounding box (x1, y1, x2, y2).
33            bbox2: Current bounding box (x1, y1, x2, y2).
34            fps: Effective frames per second (after frame-skipping).
35            frame_width: Width of the video frame in pixels.
36        """
37        raw_velocity = get_velocity(bbox1, bbox2, fps)
38        if frame_width <= 0:
39            return raw_velocity # Fallback to raw pixels if width unknown
40        return raw_velocity / frame_width
41
42    def calculate_iou(boxA: Tuple[float, float, float, float], boxB: Tuple[float, float,
float, float]) -> float:
43        """Calculate the Intersection over Union (IoU) of two bounding boxes.
44        bbox format: (x1, y1, x2, y2)
45        """
46        xA = max(boxA[0], boxB[0])
47        yA = max(boxA[1], boxB[1])
48        xB = min(boxA[2], boxB[2])
49        yB = min(boxA[3], boxB[3])
50
51        interArea = max(0, xB - xA + 1) * max(0, yB - yA + 1)
52        if interArea == 0:
53            return 0.0
54
55        boxAArea = (boxA[2] - boxA[0] + 1) * (boxA[3] - boxA[1] + 1)
56        boxBArea = (boxB[2] - boxB[0] + 1) * (boxB[3] - boxB[1] + 1)
```

```

57
58         iou = interArea / float(boxAArea + boxBArea - interArea)
59         return iou
60

```

18. user (2026-06-18T07:48:21.152Z)

```

1      """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3      This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4      of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5      the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7      The ABC lets us:
8      - Keep the current WSL adapters working unchanged for Windows users.
9      - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10     the hybrid segmenters or the model-agnostic windowing brain.
11     - Provide a single place to evolve the healthcheck / error contract and thread
12     detector readiness into per-video reports.
13
14     Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15     - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16       Run detector, return the Windows-side path to the produced trajectory CSV
17       (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18       The caller (hybrid) is responsible for output dir; impls may stage the video
19       into WSL fs for perf.
20
21     - healthcheck() -> tuple[bool, str]
22       Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23       Message is logged (and can be surfaced to reports or UX in future).
24       Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26     Existing behavior is 100% preserved. The only additions are the inheritance and
27     the explicit common type for future implementers.
28
29     How to add a new runner (for docs / future contributors):
30     1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31     2. Implement run_predict and healthcheck (return (bool, str) tuple).
32     3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33        TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34        deliberately static and model-agnostic). If the new detector needs different
35        windowing, it can implement its own or we generalize later.
36     4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37        registry never pulls torch/tf on a machine that doesn't have them).
38     5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39        DetectorRunner instances (they already do the right thing via duck-typing + the
40        type hints we add here).
41     6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42        returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43     7. No change to eval/calibration code paths (they consume the CSV + the static
44        windowing helpers directly).
45
46     Optional Linux-native path (the main de-risk goal):
47     A LinuxNativeRunner (or ONNXRunner) would:
48     - Skip all wsl / _stage / conda activate.
49     - Load weights locally (torch or onnx) or call a native binary.
50     - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51     - healthcheck can just check "weights file exists and torch importable + CUDA?".
52     This removes the WSL requirement for Linux users / future SaaS workers while
53     keeping the exact same candidate quality characteristics (or better, once native
54     weights are available).
55
56     Error surfacing into reports:
57     The healthcheck message is already logged by the hybrids on failure. The reporting
58     harvest sees the outcome as "0 candidates" + detector name. When we wire richer

```

```

59     segmenter Results (or pass detector_health into emit_indexer_report context), the
60     (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61     stage. The ABC makes that future change local to one place.
62
63     See also:
64     - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65     - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66     - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67     """
68
69     import logging
70     import shlex
71     import subprocess
72     from abc import ABC, abstractmethod
73     from typing import Any, Callable, List, Optional, Tuple
74
75     logger = logging.getLogger(__name__)
76
77
78     def wsl_tilde_quote(path: str) -> str:
79         """shlex.quote that still lets bash expand a leading `~/`.
80
81         ``shlex.quote("~/clips/x.mp4")`` single-quotes the whole path, so bash
82         treats ``~`` literally and ``cp``/``mkdir`` fail with "No such file or
83         directory" ? this broke live WSL staging for the default ``~/clips``
84         stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
85         "shlex-quote breaks ~ expansion" item). Quoting only the part after
86         ``~/`` keeps expansion AND protects spaces/specials in the remainder.
87         """
88         if path == "~":
89             return path
90         if path.startswith("~/"):
91             return "~/ " + shlex.quote(path[2:])
92         return shlex.quote(path)
93
94
95     class WslCommandMixin:
96         """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.
97
98         Lives beside (not on) DetectorRunner: a future Linux-native runner must not
99         inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
100         and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
101         ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
102         """
103
104         cfg: Any
105
106         def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
107             full = f"source {self.cfg.conda_sh} && {inner_cmd}"
108             argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
109             logger.debug("WSL exec: %s", full)
110             return subprocess.run(argv, capture_output=True, text=True,
111                                   timeout=(self.cfg.timeout_sec or None))
112
113         # ---- cache hygiene helpers (shared by both WSL runners) -----
114         def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
115             """Best-effort free space (GB) on the WSL filesystem holding ``wsl_path``.
116
117             Returns None if it can't be determined (never raises). Used as a pre-run
118             disk guard so a long extraction doesn't fill the disk mid-flight.
119             """
120             try:
121                 res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
122                 'NR==2{{print $4}}'")

```

```

123         return None
124     try:
125         return int((res.stdout or "").strip()) / (1024 ** 3)
126     except (ValueError, AttributeError):
127         return None
128
129 def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
130     """Best-effort recursive delete of WSL paths (post-success cleanup; never
131 raises)."""
132     paths = [p for p in wsl_paths if p]
133     if not paths:
134         return
135     quoted = " ".join(wsl_tilde_quote(p) for p in paths)
136     try:
137         self._wsl_bash(f"rm -rf {quoted}")
138     except (subprocess.TimeoutExpired, OSError) as e:
139         logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
140
141 def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
142     """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.
143
144     Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
145     filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path
146     fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
147     trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
148     A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE
149     any
150     frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
151     ``cp`` still gets its chance (we never block a genuinely-readable source).
152     """
153     try:
154         res = self._wsl_bash(f"test -r {shlex.quote(src_mnt)} && echo READABLE")
155     except (subprocess.TimeoutExpired, OSError):
156         return None
157     if "READABLE" in (res.stdout or ""):
158         return None
159     return (
160         f"video source is not readable from WSL at '{src_mnt}'. Google Drive, UNC, "
161         f"and other network/virtual filesystems are invisible to WSL's /mnt ? copy "
162         f"the video to a local disk (e.g. C:\\ or the WSL filesystem) and retry."
163     )
164
165 class DetectorRunner(ABC):
166     """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
167
168     @abstractmethod
169     def run_predict(self, video_win_path: str, output_win_dir: str,
170                    video_id: Optional[str] = None) -> Optional[str]:
171         """Run inference and return path to trajectory CSV (or None on failure).
172
173         ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
174         videos that happen to share a filename cannot reuse each other's frames or CSV.
175         It is optional for back-compat; callers in the live pipeline always pass it.
176         """
177         pass
178
179     @abstractmethod
180     def healthcheck(self) -> Tuple[bool, str]:
181         """Quick check if the runner (env, weights, etc.) is usable.
182
183         Returns (ok, message). Never raises. Cheap to call before a long run.
184         """
185         pass

```

```

186         # De facto trajectory contract: the trajectory hybrids call these
187         # unconditionally after run_predict. Concrete runners reuse/re-export the
188         # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
189         # a runner with custom windowing supplies its own with the same shapes.
190         parse_trajectory_csv: Callable[..., Any]
191         trajectory_to_action_windows: Callable[..., Any]
192

```

19. assistant (2026-06-18T07:48:33.843Z)

Now let me carefully analyze the most suspicious spots. Let me trace through `\_blob\_fallback\_chop` for a potential infinite loop and the midpoint chop logic when frames are dense.

Let me trace `\_split\_overlong\_groups` hard_cap and `\_blob\_fallback\_chop` for the case where the midpoint chop can fail to make progress. The `split\_i = min(range(1, len(h)), key=...)` returns an index in `[1, len(h))`. So `h[:split\_i]` has at least 1 element and `h[split\_i:]` has at least 1 element. Both are strictly smaller than `h` (since split_i >= 1 and split_i <= len(h)-1). So progress is guaranteed. Good.

Now, the key concern: in `\_split\_overlong\_groups` hard_cap midpoint chop ? `split\_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))`. This minimizes distance to the temporal midpoint. But consider a group where the frames are NOT evenly distributed. The midpoint could land such that `split\_i` puts almost everything on one side. Let me check: could the split produce a piece that STILL exceeds the cap but is identical in span to the parent, causing non-termination? No ? split_i in [1, len-1] always reduces element count. Each piece has fewer elements, so it terminates. Fine.

Let me look more carefully at `\_trim\_inactive\_tail`. The sliding window: `while frames[lo] <= frames[i] - trail\_frames`. Let me verify the window semantics and a potential issue with the `min\_density` threshold and the first frame.

Let me check the `two\_sided\_motion` boundary: a person with `coord == pos` is assigned to `far`. And `point\_in\_polygon` for a foot exactly on the net. These are semantic choices, not bugs.

Let me now look at a real concern in `detections\_over\_windows`: the `progress\_cb` and the loop `while frame\_idx <= max\_end`. Let me check `max\_end = max(ef for \_, ef in windows)`. If a window has `end\_frame` beyond video length, the `grab()` returns False and breaks. Fine.

Let me examine the `wasb\_infer.py` resume logic ? `load\_and\_clean\_cache` and the `det\_by\_fid` accumulation when resuming. Let me check that carefully.

20. user (2026-06-18T07:48:35.024Z)

```

1      """Resumable-cache layer of wasb_infer (pure-Python parts; no WSL/GPU).
2
3      Covers the reboot-resume guarantees: per-window incremental cache, cross-window
4      detection accumulation on replay, crash-truncation recovery, manifest
5      compatibility/invalidation, and atomic trajectory writes.
6
7      Also covers the streaming fast-path's pure helpers (synthetic frame addressing,
8      window builder, the sliding-buffer frame store, and the read_image shim) ? these
9      let the detector read frames straight from the decoded video instead of from ~46k
10     PNGs on disk, while preserving output exactly (byte-identical, verified).
11     """
12     import json
13     import os
14     import os.path as osp
15
16     import numpy as np
17     import pytest
18
19     from backend.pipeline.detectors import wasb_infer as wi
20
21

```

```

22 def _win(idx, frames):
23     """A window record: frames = [(fid, [[x,y,score,scale], ...]), ...]."""
24     return {"w": idx, "f": [[fid, dets] for fid, dets in frames]}
25
26
27 def _write_windows(cache_dir, windows):
28     det_jsonl, _ = wi._cache_paths(cache_dir)
29     with open(det_jsonl, "a") as f:
30         wi._append_windows(f, windows)
31
32
33 # ----- #
34 # detection (de)serialization
35 # ----- #
36 def test_det_plain_and_back_roundtrip():
37     det = {"xy": np.array([439.0, 64.5]), "score": np.float32(0.87), "scale": 0}
38     plain = wi._det_plain(det)
39     assert plain == [439.0, 64.5, float(np.float32(0.87)), 0]
40
41     back = wi._dets_to_tracker([plain])
42     assert len(back) == 1
43     assert isinstance(back[0]["xy"], np.ndarray) # tracker does vector math on xy
44     assert back[0]["xy"].tolist() == [439.0, 64.5]
45     assert back[0]["scale"] == 0
46
47
48 # ----- #
49 # cache replay + resume index
50 # ----- #
51 def test_replay_accumulates_detections_across_windows(tmp_path):
52     cache = str(tmp_path / "c")
53     os.makedirs(cache)
54     # frame 1 appears in BOTH windows -> its candidates must accumulate in window order;
55     # frame 2 is empty in window 0 but detected in window 1.
56     _write_windows(cache, [
57         _win(0, [(0, [[10, 20, 0.9, 0]]), (1, [[11, 21, 0.8, 0]]), (2, [])]),
58         _win(1, [(1, [[12, 22, 0.7, 0]]), (2, [[13, 23, 0.6, 0]]), (3, [])]),
59     ])
60     windows_done, det = wi.load_and_clean_cache(cache)
61     assert windows_done == 2
62     assert det[0] == [[10, 20, 0.9, 0]]
63     assert det[1] == [[11, 21, 0.8, 0], [12, 22, 0.7, 0]] # window-0 then window-1
64     assert det[2] == [[13, 23, 0.6, 0]]
65     assert det[3] == [] # seen but never detected
66
67
68 def test_resume_index_equals_window_count(tmp_path):
69     cache = str(tmp_path / "c")
70     os.makedirs(cache)
71     _write_windows(cache, [_win(i, [(i, [[i, i, 0.5, 0]])]) for i in range(5)])
72     windows_done, _ = wi.load_and_clean_cache(cache)
73     assert windows_done == 5
74
75
76 def test_empty_cache(tmp_path):
77     cache = str(tmp_path / "c")
78     os.makedirs(cache)
79     windows_done, det = wi.load_and_clean_cache(cache)
80     assert windows_done == 0 and dict(det) == {}
81
82
83 # ----- #
84 # crash robustness
85 # ----- #
86 def test_truncated_trailing_line_recovered_and_cleaned(tmp_path):

```

```

87     cache = str(tmp_path / "c")
88     os.makedirs(cache)
89     _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
90                          _win(1, [(1, [[2, 2, 0.8, 0]])]))
91     det_jsonl, _ = wi._cache_paths(cache)
92     with open(det_jsonl, "a") as f:                # simulate a crash mid-flush
93         f.write({'w':2,"f":[[2,[[3,3,']
94     windows_done, det = wi.load_and_clean_cache(cache)
95     assert windows_done == 2                        # only the two clean windows
survive
96     assert det[1] == [[2, 2, 0.8, 0]]
97     # file is rewritten to the clean prefix -> appending window 2 stays contiguous
98     with open(det_jsonl) as f:
99         lines = [json.loads(ln) for ln in f if ln.strip()]
100    assert [o["w"] for o in lines] == [0, 1]
101    _write_windows(cache, [_win(2, [(2, [[3, 3, 0.7, 0]])]))
102    assert wi.load_and_clean_cache(cache)[0] == 3
103
104
105    def test_noncontiguous_window_index_stops_replay(tmp_path):
106        cache = str(tmp_path / "c")
107        os.makedirs(cache)
108        _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]])]),
109                             _win(5, [(5, [[2, 2, 0.8, 0]])])) # gap -> stop after w=0
110        windows_done, det = wi.load_and_clean_cache(cache)
111        assert windows_done == 1
112        assert 5 not in det
113
114
115    # ----- #
116    # manifest: atomic write + compatibility / invalidation
117    # ----- #
118    def test_manifest_roundtrip_and_compat(tmp_path):
119        cache = str(tmp_path / "c")
120        os.makedirs(cache)
121        m = {
122            "version": wi.CACHE_VERSION,
123            "frames_dir": osp.abspath("/x/frames"),
124            "weights": "w.pth.tar",
125            "sport": "badminton",
126            "frames_in": 3,
127            "frames_total": 100,
128            "windows_total": 98,
129            "windows_done": 42,
130        }
131        wi.write_manifest(cache, m)
132        assert not osp.exists(osp.join(cache, wi._MANIFEST_FILE + ".tmp")) # atomic, no
tmp left
133        assert wi.read_manifest(cache) == m
134
135        base = dict(frames_dir="/x/frames", weights="w.pth.tar", sport="badminton",
136                    frames_in=3, frames_total=100)
137        assert wi.manifest_compatible(m, **base)
138        assert not wi.manifest_compatible(m, **{**base, "frames_total": 101}) # more
frames
139        assert not wi.manifest_compatible(m, **{**base, "weights": "other.pth"})
140        assert not wi.manifest_compatible(m, **{**base, "sport": "tennis"})
141        assert not wi.manifest_compatible({}, **base) # no cache
142        assert not wi.manifest_compatible(**m, "version": 999, **base) # schema bump
143
144
145    def test_read_manifest_missing_returns_none(tmp_path):
146        assert wi.read_manifest(str(tmp_path)) is None
147
148

```



```

149 def test_reset_cache_removes_files(tmp_path):
150     cache = str(tmp_path / "c")
151     os.makedirs(cache)
152     _write_windows(cache, [_win(0, [(0, [[1, 1, 0.9, 0]]))]])
153     wi.write_manifest(cache, {"version": wi.CACHE_VERSION})
154     wi.reset_cache(cache)
155     det_jsonl, mpath = wi._cache_paths(cache)
156     assert not osp.exists(det_jsonl) and not osp.exists(mpath)
157
158
159 # ----- #
160 # misc helpers
161 # ----- #
162 def test_default_cache_dir_is_next_to_output():
163     out = osp.join("some", "dir", "sample_long_traj.csv")
164     cd = wi.default_cache_dir(out)
165     assert osp.basename(cd) == "sample_long_traj_wasbcache"
166     assert osp.dirname(cd) == osp.dirname(osp.abspath(out))
167
168
169 def test_atomic_write_trajectory(tmp_path):
170     out = str(tmp_path / "out" / "traj.csv")
171     wi._atomic_write_trajectory(out, [(0, 1, 439.0, 64.0), (1, 0, -1, -1)])
172     assert not osp.exists(out + ".tmp")
173     with open(out) as f:
174         text = f.read().splitlines()
175     assert text[0] == "Frame,Visibility,X,Y"
176     assert text[1] == "0,1,439.0,64.0"
177     assert text[2] == "1,0,-1,-1"
178
179
180 # ----- #
181 # streaming fast-path: synthetic frame addressing + windowing (pure)
182 # ----- #
183 def test_synthetic_frame_path_roundtrip():
184     # The synthetic name must match the on-disk scheme so _frame_id parses the SAME id.
185     assert wi.synthetic_frame_path(0) == "00000000.png"
186     assert wi.synthetic_frame_path(180) == "00000180.png"
187     for i in (0, 1, 7, 180, 46123):
188         assert wi._index_from_synthetic_path(wi.synthetic_frame_path(i)) == i
189         # disk-style names parse to the same id (output-preservation invariant)
190         assert wi._frame_id(f"frame_{i:08d}.png") == i
191
192
193 def test_build_windows_matches_disk_logic():
194     frames = [wi.synthetic_frame_path(i) for i in range(6)]
195     # frames_in=3 -> 4 sliding windows, each 3 consecutive, overlapping by 2.
196     wins = wi.build_windows(frames, 3)
197     assert len(wins) == 4
198     assert wins[0] == frames[0:3]
199     assert wins[1] == frames[1:4]
200     assert wins[-1] == frames[3:6]
201
202
203 def test_build_windows_too_few_frames_is_empty():
204     assert wi.build_windows(["00000000.png", "00000001.png"], 3) == []
205
206
207 # ----- #
208 # streaming fast-path: SequentialFrameStore (sliding buffer; pure)
209 # ----- #
210 def _counting_reader():
211     """A fake forward-only decoder: returns ('frame', i) and records call order."""
212     calls = []
213

```

```

214     def reader(i):
215         calls.append(i)
216         return ("frame", i)
217
218     return reader, calls
219
220
221 def test_store_reads_each_frame_once_in_order_for_sliding_windows():
222     # The detector's real access pattern: ImageDataset reads window i's frames in order
223     # (i, i+1, i+2), and samples are requested i=0,1,2,... So the global read sequence
224     # DIPS BACK by window-1 at each boundary: 0,1,2, 1,2,3, 2,3,4. Every source frame
225     # must still be decoded exactly once (the whole point of the in-memory buffer).
226     reader, calls = _counting_reader()
227     store = wi.SequentialFrameStore(reader, total=5, window=3)
228     order = [0, 1, 2, 1, 2, 3, 2, 3, 4]
229     got = [store.get(i) for i in order]
230     assert got == [("frame", i) for i in order]
231     assert calls == [0, 1, 2, 3, 4] # decoded once each, in order
232
233
234 def test_store_evicts_outside_window_bounded_memory():
235     # window=3 -> buffer holds at most 3 frames; frames older than max_seen-2 are
dropped.
236     reader, _ = _counting_reader()
237     store = wi.SequentialFrameStore(reader, total=100, window=3)
238     for i in (0, 1, 2, 3, 4):
239         store.get(i)
240     assert len(store._buf) <= 3
241     # frame 1 is now below the floor (max_seen=4, floor=2) -> evicted, re-request
raises.
242     with pytest.raises(KeyError):
243         store.get(1)
244
245
246 def test_store_keeps_window_minus_one_lookback():
247     # The legal dip-back of exactly window-1 must NOT raise (it's the real access
pattern).
248     reader, _ = _counting_reader()
249     store = wi.SequentialFrameStore(reader, total=10, window=3)
250     store.get(0)
251     store.get(1)
252     store.get(2) # max_seen=2, floor=0 -> frames 0,1,2 all retained
253     assert store.get(1) == ("frame", 1) # window boundary look-back is fine
254     assert store.get(2) == ("frame", 2)
255
256
257 def test_store_resume_starts_at_start_index_without_decoding_prefix():
258     # On a resume the first needed frame is `start_index`; the prefix must NOT be
decoded.
259     reader, calls = _counting_reader()
260     store = wi.SequentialFrameStore(reader, total=10, window=3, start_index=4)
261     store.get(4)
262     store.get(5)
263     assert calls == [4, 5] # frames 0..3 skipped entirely (seek, not decode)
264     with pytest.raises(KeyError): # below the resume floor -> never available
265         store.get(3)
266
267
268 def test_store_out_of_range_raises():
269     reader, _ = _counting_reader()
270     store = wi.SequentialFrameStore(reader, total=3, window=3)
271     with pytest.raises(KeyError):
272         store.get(3)
273
274

```

```

275 # ----- #
276 # streaming fast-path: read_image shim resolves synthetic paths to in-memory RGB
277 # frames (WASB reads via PIL read_image, NOT cv2.imread) + falls through otherwise.
278 # ----- #
279 def _bgr_frame(b, g, r, h=4, w=4):
280     cv2 = pytest.importorskip("cv2") # noqa: F841 (importorskip gates the suite on cv2)
281     frame = np.zeros((h, w, 3), dtype=np.uint8)
282     frame[..., 0], frame[..., 1], frame[..., 2] = b, g, r
283     return frame
284
285
286 def test_make_streamed_read_image_returns_pil_rgb_and_falls_through():
287     cv2 = pytest.importorskip("cv2")
288
289     # Loader yields a distinct-channel BGR frame per index (B=10+i, G=20+i, R=30+i).
290     def loader(i):
291         return _bgr_frame(10 + i, 20 + i, 30 + i)
292
293     store = wi.SequentialFrameStore(loader, total=4, window=1)
294     shim = wi._make_streamed_read_image(store, real_read_image=lambda p, *a, **k:
("REAL", p))
295
296     img = shim(wi.synthetic_frame_path(0))
297     arr = np.asarray(img)
298     # The shim must return an RGB array (channels swapped from the BGR loader frame),
matching
299     # what WASB's read_image (PIL Image.open().convert('RGB')) yields off disk ? NOT raw
BGR.
300     assert arr.shape == (4, 4, 3)
301     assert (arr[..., 0] == 30).all() # R
302     assert (arr[..., 1] == 20).all() # G
303     assert (arr[..., 2] == 10).all() # B
304     assert np.array_equal(arr, cv2.cvtColor(loader(0), cv2.COLOR_BGR2RGB))
305
306     # A path with no parseable index falls through to the real reader.
307     assert shim("no-digits-here") == ("REAL", "no-digits-here")
308
309
310 def test_make_streamed_read_image_resume_primes_start_index():
311     pytest.importorskip("cv2")
312     calls = []
313
314     def loader(i):
315         calls.append(i)
316         return _bgr_frame(1, 2, 3)
317
318     store = wi.SequentialFrameStore(loader, total=10, window=1, start_index=5)
319     shim = wi._make_streamed_read_image(store, real_read_image=lambda p: None)
320     shim(wi.synthetic_frame_path(5))
321     assert calls == [5] # resume seeked to 5; frames 0..4 never decoded
322
323
324 def test_bgr_to_pil_rgb_matches_disk_roundtrip(tmp_path):
325     """Regression lock on the output-preservation proof: the streaming frame conversion
326     (`_bgr_to_pil_rgb`) is byte-identical to WASB's disk path
327     (`cv2.imwrite` PNG -> `Image.open().convert('RGB')`)."""
328     cv2 = pytest.importorskip("cv2")
329     Image = pytest.importorskip("PIL.Image")
330
331     rng = np.random.RandomState(0)
332     frame_bgr = rng.randint(0, 256, size=(16, 24, 3), dtype=np.uint8)
333
334     # Disk path: BGR -> PNG (lossless) -> PIL RGB
335     png = osp.join(str(tmp_path), "f.png")
336     cv2.imwrite(png, frame_bgr)

```

```

337         disk_rgb = np.asarray(Image.open(png).convert("RGB"))
338
339         # Streaming path: BGR -> PIL RGB directly
340         stream_rgb = np.asarray(wi._bgr_to_pil_rgb(frame_bgr))
341
342         assert np.array_equal(disk_rgb, stream_rgb)
343

```

21. assistant (2026-06-18T07:49:06.578Z)

I have a thorough understanding now. Let me drill into a few specific concerns that I want to verify against the actual numbers.

****Concern 1: `\_trim\_inactive\_tail` two-pointer window edge.**** The condition `while frames[lo] <= frames[i] - trail\_frames`. The trailing window at frame `i` should include frames within `trail\_frames` of `frames[i]`. This evicts `lo` while `frames[lo] <= frames[i] - trail\_frames`, so kept frames satisfy `frames[lo] > frames[i] - trail\_frames`, i.e. `frames[i] - frames[lo] < trail\_frames`. That's a window of width `< trail\_frames`. Reasonable.

****Concern 2: `two\_sided\_motion` denominator.**** It divides `both / len(per\_frame)`. But `per\_frame` includes ALL sampled frames, even ones with no in-court persons. That's by design (fraction of sampled frames). OK.

****Concern 3: the `detections\_over\_windows` ? `max\_end` when a window's `end\_frame` is huge.**** Not a bug.

Let me look very carefully at a subtle issue in `\_split\_overlong\_groups` regarding the `out.sort` and the hard_cap midpoint split combined with the worklist `pop()`. Let me also examine whether `min\_window\_duration` filtering interacts wrongly with `forced\_flags` alignment after R5.

Actually ? let me re-examine a real alignment bug candidate. In `trajectory\_to\_action\_windows`, after R5 produces `groups`, `forced\_flags` aligned 1:1, the final filter is:

```

```python
return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
```

```

This is fine. But let me check: when `window\_blob\_fallback` is False, `forced\_flags = [False] \* len(groups)` is set BEFORE R5. But R4 trimming happens before that and may have changed group contents (not count). `len(groups)` is stable through R4 (it's a list comprehension mapping each group). Good ? count preserved.

Now let me check whether R4's `\_trim\_inactive\_tail` can return an EMPTY group. Look: it returns `group[:last\_dense+1]` if `last\_dense >= 0` else `group`. Since `last\_dense >= 0`, the slice has at least 1 element. And the `else` returns the whole group. So never empty. Good. But what about `\_finalize` on a single-element group ? `group[0][0]` and `group[-1][0]` are the same, duration 0, filtered out by min_window_duration. Fine.

Let me now verify a potential real off-by-one in `detections\_per\_frame` vs the trajectory frame alignment, and check the colocation uses `int(p.frame)`. Let me look at one more area ? `point\_in\_polygon` with the `yj != yi` guard already handled by the outer `(yi > y) != (yj > y)` check which guarantees `yi != yj`. So the `if yj != yi else xi` ternary's else branch is dead. Not a bug.

Let me look at the `mahadevpura` and check whether there's an issue with the `\_index\_from\_synthetic\_path` returning -1 and the `store.get`. Let me check `run\_video\_streaming` builds `frame\_list` and `\_frame\_id` parse of `00000000.png` returns 0. Good.

Let me now examine the wasb hybrid / wasb_runner caller to check the `output\_win\_dir` for the streaming `--out` path quoting. Let me check the `wasb\_runner` `--out` argument: `wsl\_out\_dir + '/' + key + '\_wasb.csv'` passed to `wsl\_tilde\_quote`. If `wsl\_out\_dir` is a `/mnt/c/...` path it's fine. Let me check a subtle bug: the `wsl\_tilde\_quote(wsl\_out\_dir + '/' + key + '\_wasb.csv')` ? if `key` contains shell-special chars they're quoted. Fine.

Let me verify the most promising lead ? the `SequentialFrameStore` eviction when `window=1`

(streaming default forced path uses `window=frames\_in`, but the resume `start\_index` interaction). Let me trace `test\_store\_resume`. Looks correct.

Let me check one concrete thing: in `run()`, when resuming, `det\_f = open(det\_jsonl, "a")` appends. But `load\_and\_clean\_cache` already rewrote the file to the clean prefix WITH a trailing newline (`"\n".join(valid) + "\n"`). Appending in "a" mode after that is contiguous. Good.

Now let me look at a potential real bug: `manifest\_compatible` checks `frames\_total` equality, but on the streaming resume path, `frames\_total = len(frames)` where `frames` was capped by `limit`. Let me verify the `limit` is applied consistently. In `run()`: `if limit: frames = frames[:limit]` then `frames\_total=len(frames)`. On resume after reboot with the same args, consistent. Fine.

Let me check `extract\_frames` idempotency bug: it compares `existing >= expected` PNG count but globs only `\*.png`. Fine.

Let me look harder at `detections\_over\_windows` for the overlapping-windows sampling grid correctness and a possible off-by-one in `progress\_cb`. Let me also re-examine the `frame\_skip` default mismatch between `extract\_action\_windows\_from\_chunk` (frame_skip recorded vs used).

Let me re-read the velocity computation in `extract\_action\_windows\_from\_chunk`. `effective\_fps = fps / max(frame\_skip, 1)`. `max\_gap\_frames = int(merge\_gap \* fps)` ? uses RAW fps, but `frame\_idx` increments by 1 each raw frame (skipped ones are `continue`d but still increment `frame\_idx`). So active_frames store RAW frame indices. The gap comparison `af[0] - group[-1][0] <= max\_gap\_frames` uses raw frame indices vs `merge\_gap \* fps` (raw). Consistent. And `start = group[0][0]/fps` uses raw fps. Consistent. Good.

Now compare to `trajectory\_to\_action\_windows`: there `merge\_gap` uses `int(merge\_gap \* fps)` and frames are raw frame indices from the CSV. Consistent.

Let me focus on verifying one thing I'm suspicious about: the `\_trim\_inactive\_tail` `min\_density` comparison and whether `last\_dense` can stay at the very first frame causing a window to collapse to length-1 incorrectly trimming a real rally. That's tuning, not a defect.

Let me now run the actual tests to confirm the baseline is green and look for anything I can reproduce.

22. user (2026-06-18T07:49:15.512Z)

```
..... [ 93%]  
..... [100%]  
77 passed in 4.62s
```

23. assistant (2026-06-18T07:49:25.369Z)

Baseline green. Now let me probe a specific suspicion: the `\_trim\_inactive\_tail` two-pointer when `trail\_frames` is small and frames are sparse, and whether the `forced\_flags` length stays aligned with `groups` if `min\_window\_duration` filters AND R5 is on. Already verified alignment.

Let me now look hard at one thing that stands out: in `\_split\_overlong\_groups`, the hard_cap midpoint chop uses `mid = (g[0][0] + g[-1][0]) / 2.0` and picks the closest frame index. But the worklist uses `work.pop()` (LIFO) and at the end does `out.sort(key=lambda grp: grp[0][0])`. That sort keys on the first frame of each group, which is unique across non-overlapping partition pieces. Fine.

Let me examine a genuinely suspicious numeric case in `\_blob\_fallback\_chop`: the midpoint split `min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid))`. Consider a blob where ALL frames are at the SAME index? Impossible (frames are distinct increasing).

Let me write a quick adversarial probe for the midpoint chop where frame distribution is skewed ? can a piece still exceed cap but split_i always lands at index 1 (or len-1), leaving the big piece nearly unchanged but still reducible? Yes, it reduces by 1 element each time, so it terminates but could produce MANY tiny pieces plus the big one needing many iterations. Not a correctness bug. Let me verify it actually caps span ? cap for a pathological distribution.

24. user (2026-06-18T07:49:30.533Z)

R5 blob-fallback: forced 2 midpoint cut(s) on continuously-active group(s) with no rally-end signal ? this clip needs rally-STATE cues (net-crossings / two-sided motion), not a bigger cap
pieces 3 max span 25 all<=30 True
partition ok True
hardcap pieces 3 max span 25 all<=30 True

25. assistant (2026-06-18T07:49:44.816Z)

Both terminate correctly and cap the span. Now let me probe the `\_trim\_inactive\_tail` two-pointer for a subtle window-width off-by-one and a fail-safe edge.

Let me also examine the genuinely suspicious area I noticed: in `detections\_over\_windows`, `fps = cap.get(...)` if `cap.isOpened()` else `0.0`. If cap is opened but `cap.get(FPS)` returns a value, `fps/step` for the tracker. But if `cap` is NOT opened, returns early. Fine.

Let me now look at a real potential issue in `person\_detector.detect\_and\_track`: `boxes[mask]` indexing. `mask = (labels == \_PERSON\_CLASS\_ID) & (scores >= score\_thresh)`. `labels`, `scores`, `boxes` come from `out["..."].detach().cpu().numpy()`. These are numpy arrays. `boxes[mask]` works if `boxes` is shape (N,4) and mask shape (N,). Good. Empty mask ? `return []`. Good.

Let me check the one place I'm most suspicious of: `two\_sided\_motion`'s `len(per\_frame)` denominator could be 0 ? but guarded by `not per\_frame`. Good.

Now let me carefully look at `shuttle\_player\_colocation` ? it uses `per\_frame.get(int(p.frame))`. But `dets` is None` check vs empty list `[]`. If a frame was sampled but had no detections, `per_frame[frame]` would be `[]` (not None), so `dets` is None` is False, `aligned += 1`, and `any(...)` over empty is False ? not colocated. Correct: a sampled-but-empty frame counts as aligned-not-colocated. Good.

Let me now examine a subtle but real concern in `detections\_per\_frame`: the tracker is created with `fps/step`, and frames are sampled at `(frame\_idx - start\_frame) % step == 0`. But it reads EVERY frame sequentially via `cap.read()` (not grab/retrieve), running detection only on sampled ones. Fine. The `frame\_idx <= end\_frame` includes `end\_frame`. Good (test confirms [0,3,6,9] for end=9).

Let me look for the real bug. Let me re-examine `\_trim\_inactive\_tail` carefully with a concrete trace where there's a gap inside the group (it operates on a single group AFTER splitting, but the group can still contain internal gaps below merge_gap). The two-pointer uses `frames[lo] <= frames[i] - trail\_frames`. Let me verify the density math is per the docstring and check a fail case.

26. user (2026-06-18T07:49:50.838Z)

kept last frame 81 len 82
all-slow kept whole: True len 100
single-fast-start kept last 2 len 3

27. assistant (2026-06-18T07:50:23.329Z)

Interesting ? the all-slow case keeps the whole group (fail-safe, documented). The single-fast-start trims to 3 frames (last_dense=2). These match the documented behavior. Note: the all-slow group keeps the whole thing even though there's no rally ? that is the documented fail-safe (only cuts, never invents; a window with no fast-dense point is kept whole). This is intentional, not a bug.

Now let me look very carefully at one area that could be a real defect ? the `wasb\_infer.run()` resume path where `windows\_done` from the cache is used to slice `samples[windows\_done:]` AND as `start\_index` for the streaming store. But there's a subtle issue: on resume, the detector loop's `win\_idx = windows\_done` and `done = windows\_done`, and frames payload appends to `det\_by\_fid` which ALREADY contains the replayed cache detections. So for windows that overlap (shared frames between cached and new windows), the accumulation is correct since cached windows < windows_done and new windows >= windows_done are disjoint window indices. Good.

But wait ? there IS a potential real issue. Let me check ``_streaming_read_image_patch`` ``start_index=windows_done``. The store's floor is ``start_index``. The first new window is window index ``windows_done``, which starts at frame ``windows_done`` and needs frames ``windows_done .. windows_done + frames_in - 1``. The store floor = ``windows_done``, so it can serve those. Good. But the DataLoader with ``batch_size`` and ``shuffle=False`` over ``remaining = samples[windows_done:]`` ? the FIRST sample is window ``windows_done`` reading frames starting at ``windows_done``. Store primed at ``windows_done``. Correct.

Let me now check a concrete suspicion in ``run()``: the progress/flush logic. ``done >= windows_total`` is used as a flush trigger and a "final" indicator. But ``done`` counts total windows processed (``done += nb``). On resume, ``done`` starts at ``windows_done`` and increments by batch. The final ``done`` should equal ``windows_total``. But what if ``nb`` (batch size from ``imgs.shape[0]``) and the manifest ``windows_done`` written... this looks consistent.

Let me step back and look at whether there's an issue with ``wasb_runner`` and ``to_wsl_mnt_path`` when ``output_win_dir`` is already absolute on the GPU machine. Not a defect.

Let me examine the one thing in ``person_detector`` that the comment flags as a gotcha ? ``_PERSON_CLASS_ID = 1``. The test confirms label 1 = person. Correct for torchvision COCO.

Let me look at the ``extract_action_windows_from_chunk`` ``track_history`` velocity when a track reappears after being pruned. ``track_history = {k:v for ... if k in current_ids}``. So a track not seen this frame is pruned. Next frame if it reappears, no ``prev_bbox`` ? no velocity spike. Documented and correct.

Let me check a potential real bug in ``detections_over_windows``: ``max_end = max(ef for _, ef in windows)``. The loop ``while frame_idx <= max_end``. But the ``progress_cb(frame_idx, max_end)``. If a window's ``end_frame`` exceeds the actual video length, the loop breaks early on ``grab()`` False, and that window simply gets fewer frames. Not a crash.

Now let me look for an actual contract/logic bug I may have glossed: in ``trajectory_to_action_windows``, the velocity uses ``prev`` = last visible point, but resets ``prev=None`` on invisible. However note ``prev = p`` is set at the END of the loop for visible points, so on a visible point after computing velocity, ``prev`` advances. Good.

Let me check the ``dframes > 0`` guard: frames are sorted ascending by ``points.sort``. But duplicate frame numbers? ``parse_trajectory_csv`` could yield two rows with the same Frame. After sort, ``dframes = p.frame - prev.frame`` could be 0 ? skipped (guarded). Good.

Let me look at one more genuinely suspect numeric: ``max_gap_frames = int(merge_gap * fps)``. With ``merge_gap=2.0, fps=30 ? 60``. The grouping condition ``af[0] - group[-1][0] <= max_gap_frames``. Standard.

I've done a thorough pass. Let me do a final targeted check on whether ``two_sided_motion`` and ``in_court_counts`` handle the ``net`` axis position semantics, and verify the ``point_in_polygon`` ray-cast ``x_cross`` division can't divide by zero (guarded by ``(yi>y)!=(yj>y)`` ensuring ``yj!=yi``). Confirmed safe.

Let me make one final check on a possible real issue: ``_finalize`` in ``trajectory_to_action_windows`` computes ``peak_velocity = max(vels)``. The ``active`` list only contains frames where ``velocity > velocity_thresh``. So ``vels`` is non-empty per group. Good. But ``_split_overlong_groups`` and R4 trim produce sub-groups ? always non-empty (verified). And R5 pieces non-empty. Good.

Let me verify there are no issues with the ``forced_flags`` when R5 is on but a group is ``len(g) < 2`` (single frame) ? it appends ``flags.append(False)``. Then ``_finalize`` on single-frame group ? duration 0 ? filtered. Aligned. Good.

I've been thorough. Let me do one last check ? the ``wasb_runner`` cleanup list and whether a failure between staging and inference leaks. On failure it returns without cleaning (documented as intentional for resumability). Not a defect.

Let me verify my understanding of one final potential off-by-one: ``SequentialFrameStore.get`` eviction ``new_floor = max(self._floor, self._max_seen - (self._window - 1))``. With ``window=1, new_floor = max(floor, max_seen)``. So it evicts everything below ``max_seen``, keeping only the

current frame. For window=1 that's correct (no lookback). Test
`test\_store\_reads\_each\_frame\_once` uses window=3. Confirmed by passing tests.

I've completed a comprehensive adversarial review of all the named files. The code in this area is heavily tested (77 passing tests across the four test files) and has clearly been through multiple hardening sweeps. The windowing logic (R3/R4/R5), frame/time conversions, array bounds, and GPU/np guards are all correct and well-guarded. I found no real, reachable defects.

Let me do one final verification on a subtle edge I want to be certain about ? the
`\_trim\_inactive\_tail` two-pointer when `trail\_frames` is exactly equal to a frame gap, to rule out an off-by-one that mis-densities. And confirm the `min\_density` boundary `>=`.

28. user (2026-06-18T07:50:29.612Z)

```
last 3
0 dens 1.0
1 dens 1.0
2 dens 0.6666666666666666
3 dens 0.5
4 dens 0.25
5 dens 0.0
```

29. assistant (2026-06-18T07:50:41.691Z)

The cross-check confirms the two-pointer matches a brute-force recomputation exactly: density at i=3 is 0.5 ($\geq$ min_density 0.5, dense), at i=4 drops to 0.25, so `last\_dense=3` and the function trims to frame 3. The window semantics (`frames[j] > frames[i] - trail\_frames`) and the ` $\geq$ ` boundary are implemented correctly.

I have completed a thorough adversarial review of all named files in
`backend/pipeline/detectors`:

- **tracknet_runner.py** ? R3/R4/R5 windowing (`\_split\_overlong\_groups`, `\_trim\_inactive\_tail`, `\_blob\_fallback\_chop`), frame/time conversions, the gap-grouping and finalize math. Verified termination of all recursive/iterative choppers, partition-preservation (only cuts), `forced\_flags` 1:1 alignment, non-empty group invariants, and the two-pointer density window against a brute-force recomputation.
- **wasb_infer.py** ? resumable cache contiguity/accumulation, `SequentialFrameStore` eviction floor + resume start_index, synthetic-path round-trip, manifest compatibility, atomic writes, streaming shim. All guards correct.
- **person_detector.py** ? COCO person==1 filter, mask indexing with empty-mask guard, velocity windowing with raw-frame-index/fps consistency, stale-track pruning, single-pass `detections\_over\_windows` dispatch and `detections\_per\_frame` sampling grid (inclusive of end_frame).
- **fusion_features.py** ? ray-cast point-in-polygon (division guarded), foot/center normalization with w/h<=0 guards, two-sided/colocation/motion denominators all guarded against empty inputs, sampled-but-empty vs None distinction correct.
- **wasb_runner.py** ? path translation, namespacing, staging/cleanup ordering.

All 77 tests across the four test files pass, and the code shows consistent, well-documented guarding from prior hardening sweeps. I found no real, reachable defects (no crash on reachable input, no inverted condition, no off-by-one, no unguarded np/array access, no resource leak, no contract mismatch). Reporting an empty candidate list.

30. user (2026-06-18T07:50:42.445Z)

Structured output provided successfully

31. assistant (2026-06-18T07:50:55.538Z)

I completed a thorough adversarial review of the detector area and found no real, reachable defects. Reporting an empty candidate list.

Files reviewed (all absolute paths):

- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/pipeline/detectors/tracknet\_runner.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/pipeline/detectors/wasb\_infer.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/pipeline/detectors/person\_detector.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/pipeline/detectors/fusion\_features.py`
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base/backend/pipeline/detectors/wasb\_runner.py`

What I verified specifically (the highest-risk spots in the prompt):

- R3/R4/R5 windowing in `tracknet\_runner.py`: all three choppers (`\_split\_overlong\_groups` hard-cap midpoint, `\_trim\_inactive\_tail` O(n) two-pointer, `\_blob\_fallback\_chop`) terminate, only cut (never invent/drop coverage ? confirmed via partition checks on skewed/pathological inputs), keep every emitted group non-empty, and keep `forced\_flags` aligned 1:1 with groups through R4. The trim's trailing-window density math matches a brute-force recomputation exactly, including the `>=` boundary.
- Frame/time conversions: raw-frame-index vs `merge\_gap\*fps` and `start=frame/fps+chunk\_start` are internally consistent in both `trajectory\_to\_action\_windows` and `extract\_action\_windows\_from\_chunk`; `dframes>0` guards duplicate/zero-delta frames.
- np/array guards in `person\_detector.detect\_and\_track`: empty-mask early return before `boxes[mask]`; tracker None-id skip.
- `fusion\_features.py`: ray-cast division guarded by the crossing test, all denominators guarded against empty inputs, w/h<=0 handled, sampled-but-empty vs unsampled (`None`) frames distinguished correctly in colocation.
- `wasb\_infer.py` resume/streaming: cache contiguity + cross-window accumulation, `SequentialFrameStore` floor/eviction and resume `start\_index`, manifest invalidation, atomic writes.

All 77 tests across `test\_tracknet\_runner.py`, `test\_fusion\_features.py`, `test\_person\_detector.py`, and `test\_wasb\_cache.py` pass on the pinned worktree.